机器视觉技术

人工智能学院
南开大学

机器人与信息自动化研究所
Institute of Robotics & Automatic Information System

南開大學
Nankai University

第三部分 深度学习图像处理

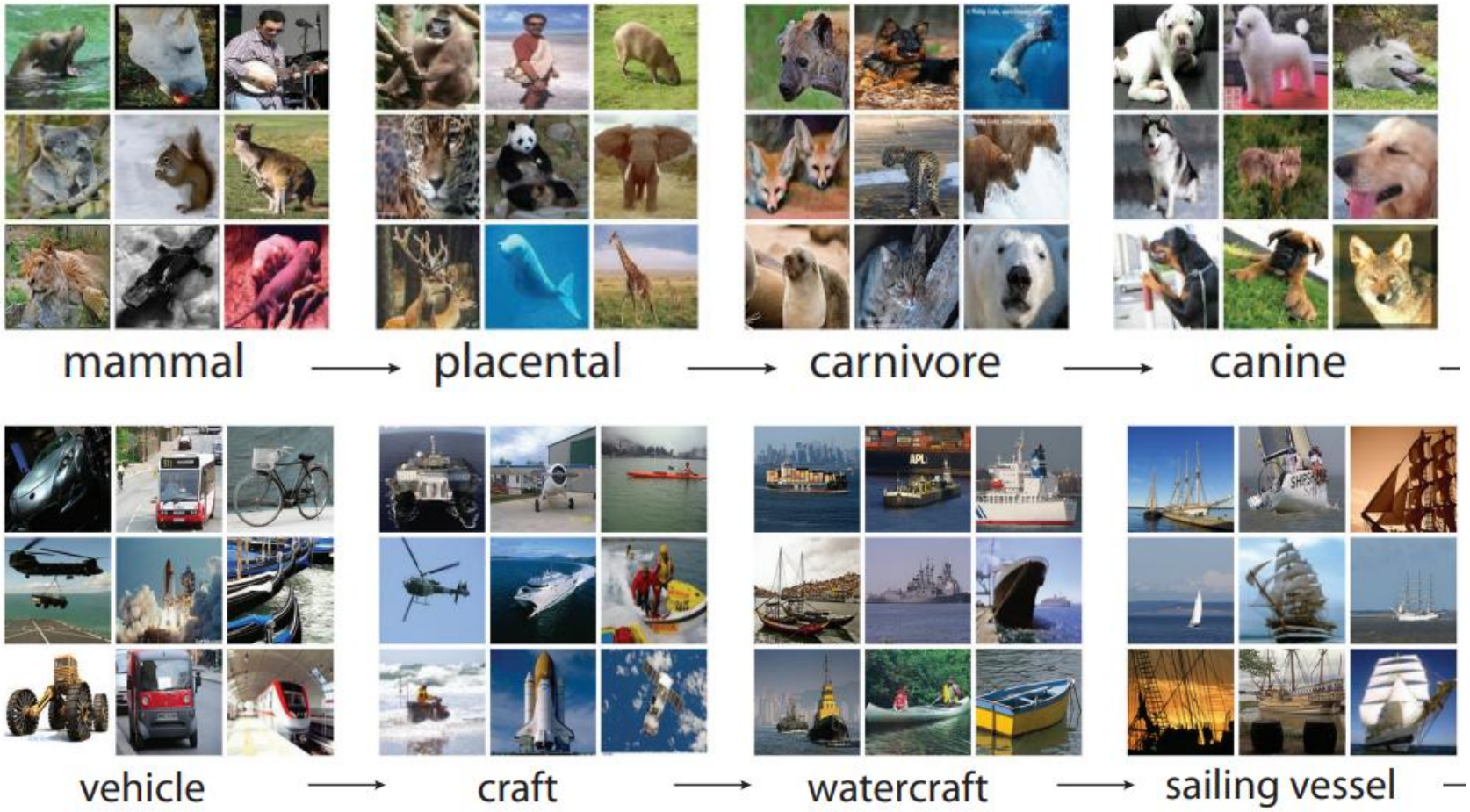
Deep Learning for Image Processing

主要内容

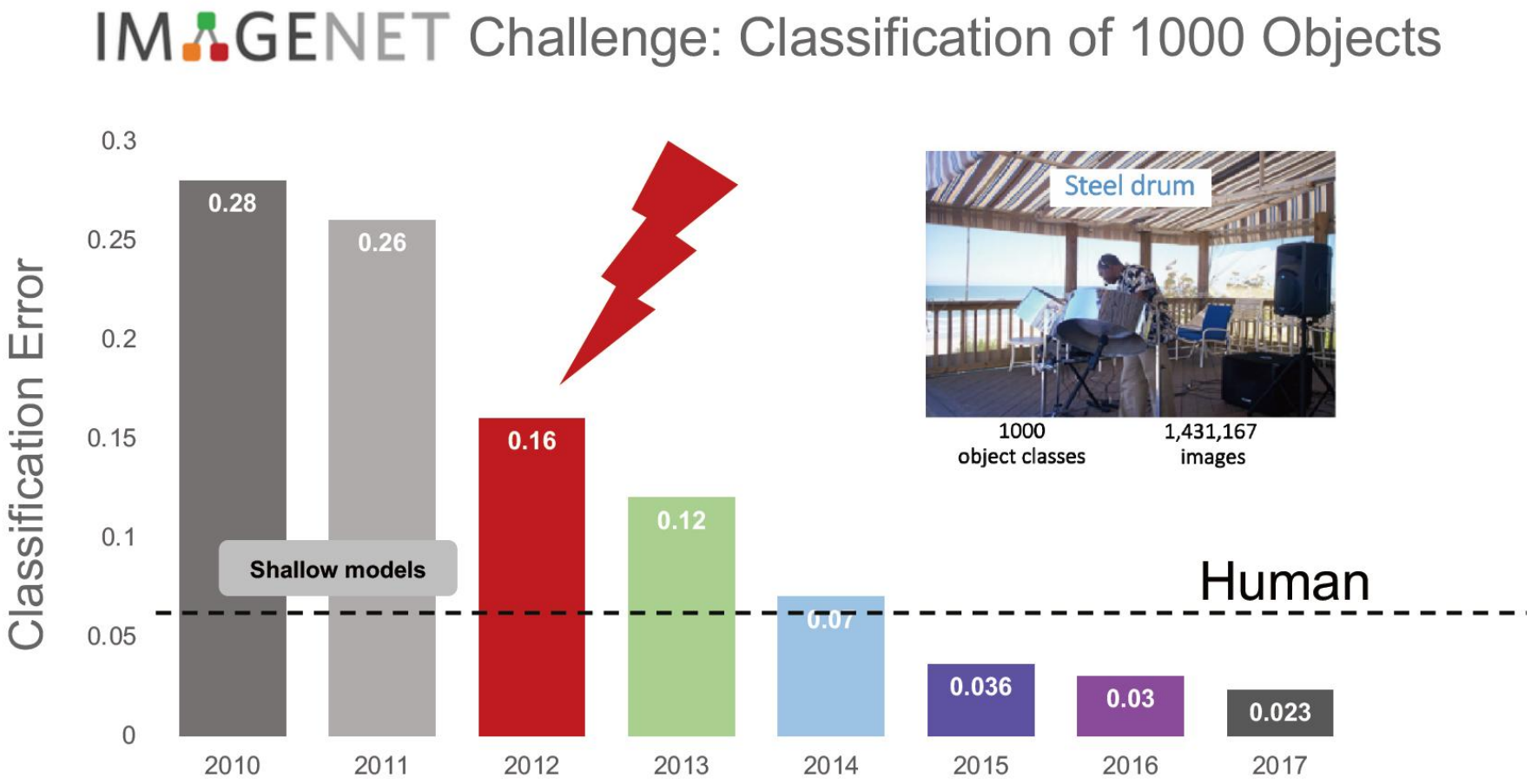
- **第六章 卷积神经网络**
- **第七章 图像分类**
  - 7.1 ImageNet挑战赛
  - 7.2 经典图像分类网络
    - AlexNet、VGGNet、GoogLeNet、ResNet
  - 7.3 迁移学习
- **第八章 目标检测与图像分割**
- **第九章 Transformer与多模态模型**

7.1 ImageNet挑战赛

训练集：1,281,167张已标注图像
验证集：50,000张已标注图像
测试集：100,000张未标注图像

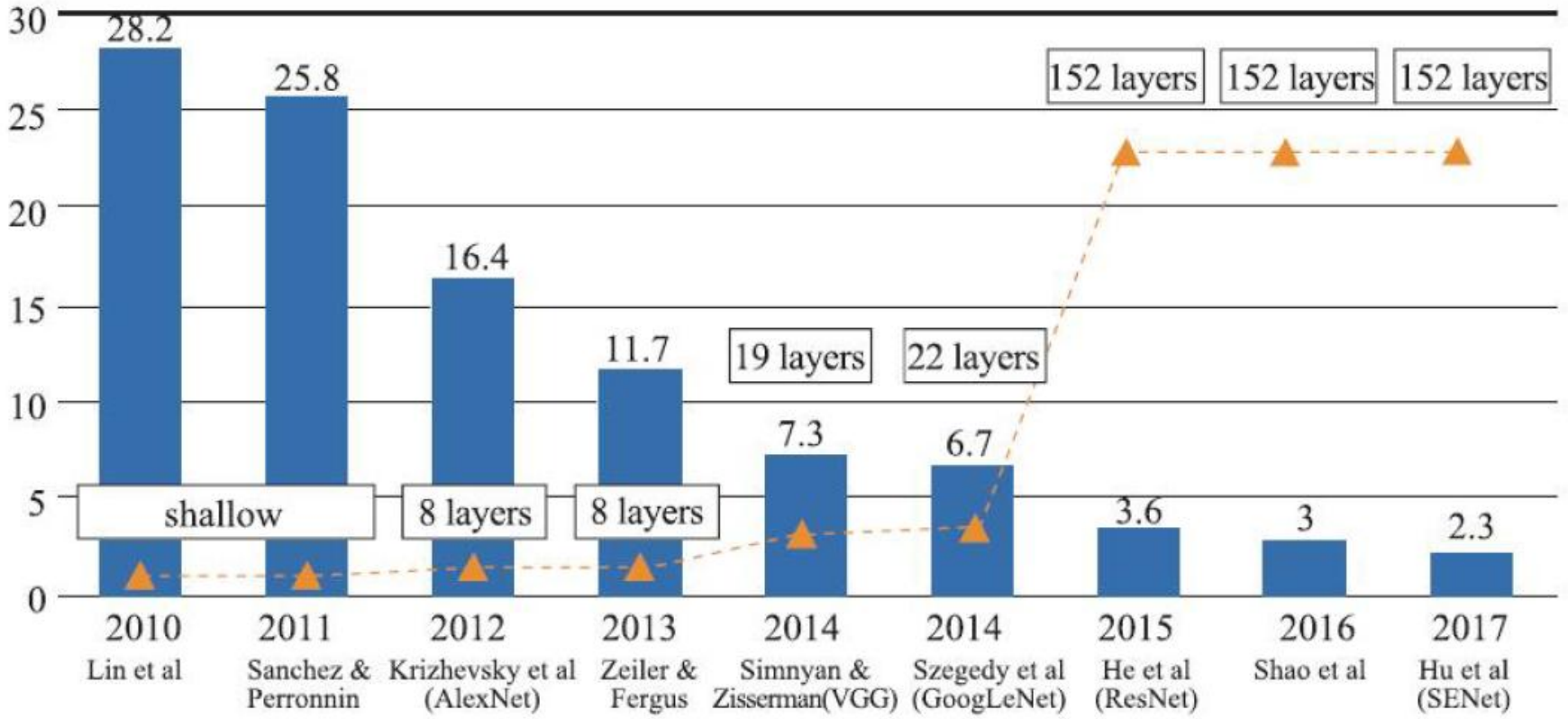


7.1 ImageNet挑战赛



Deng, J. et al. Fei-Fei, L. *CVPR*, 2009; Russakovsky et al. Fei-Fei, J. *IJCV*, 2012;

7.1 ImageNet挑战赛



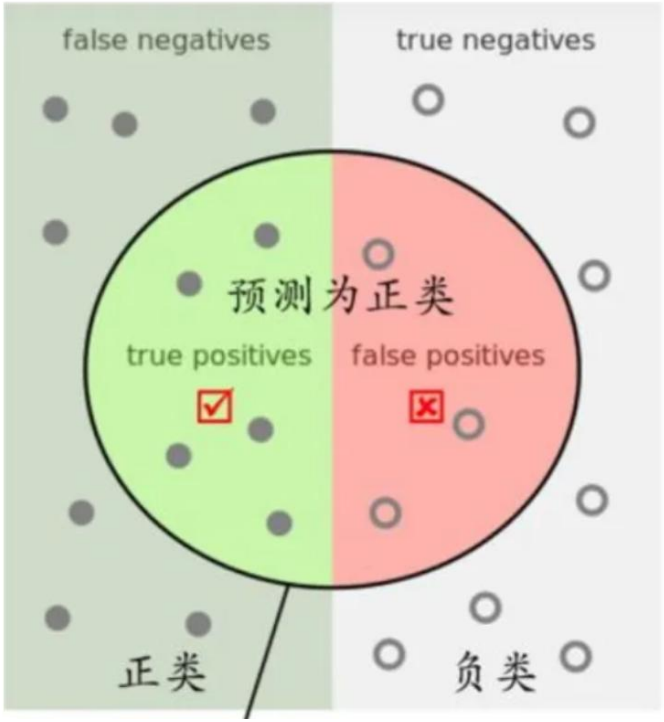

```
INPUT -> [[CONV -> RELU]*N -> POOL?]*M -> [FC -> RELU]*K -> FC
```

评价指标简介

Positive/Negative：预测为正负样本
True/false：预测是否正确

混淆矩阵

TP(True Positive)=真阳性； FP(False Positive)=假阳性
FN(False Negative)=假阴性； TN(True Negative)=真阴性



- TP+FP+TN+FN：样本总数
- TP+FN：实际正样本数
- FP+TN：实际负样本数
- TP+FP：预测结果为正样本的总数，包括预测正确的和错误的
- TN+FN：预测结果为负样本的总数，包括预测正确的和错误的

评价指标简介

Positive/Negative：预测为正负样本
True/false：预测是否正确

- **混淆矩阵**

  TP(True Positive)=真阳性； FP(False Positive)=假阳性

  FN(False Negative)=假阴性； TN(True Negative)=真阴性

- **评价指标**
  - 精确率（查准率）：模型预测为正的样本中，有多少是真的正例？用于衡量模型预测结果的准确性或可信度

    $$Precision = TP/(TP + FP)$$

  - 召回率（查全率）：所有真实的正样本中，被模型成功找出来的比例是多少？用于衡量模型发现目标的全面性或覆盖率

    $$Recall = TP/(TP + FN)$$

  - 在绝大多数情况下，精确率和召回率是相互制约

- TP+FP：预测结果为正样本的总数，包括预测正确的和错误的
- TP+FN：实际正样本数

评价指标简介

Positive/Negative：预测为正负样本
True/false：预测是否正确

- **混淆矩阵**

  TP(True Positive)=真阳性； FP(False Positive)=假阳性

  FN(False Negative)=假阴性； TN(True Negative)=真阴性

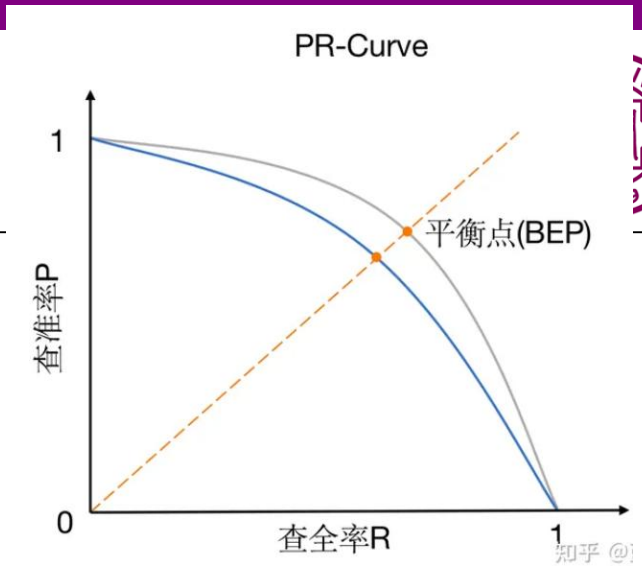
- **评价指标**
  - 精确率（查准率） Precision = TP/(TP + FP)
  - 召回率（查全率） Recall = TP/(TP + FN)
  - 准确率：模型预测正确的数据占总数据的比例

$$Acc = \frac{\mathrm{TP+TN}}{\mathrm{TP+FP+TN+FN}}$$

  - F1分数 (F1-Score)：精确率和召回率的调和平均数

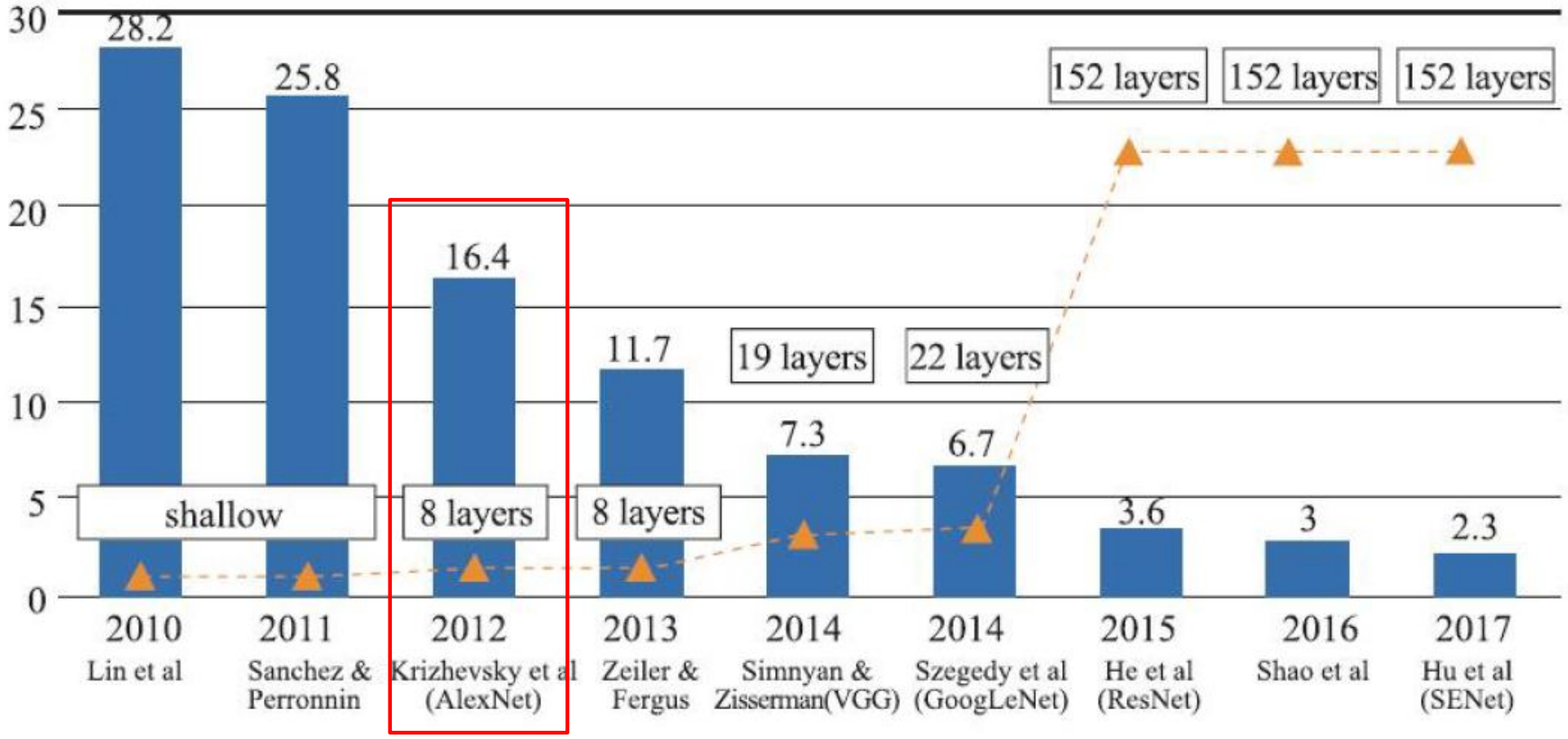
$$F1 = \frac{2*\mathrm{Precision}*\mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$$

评价指标简介

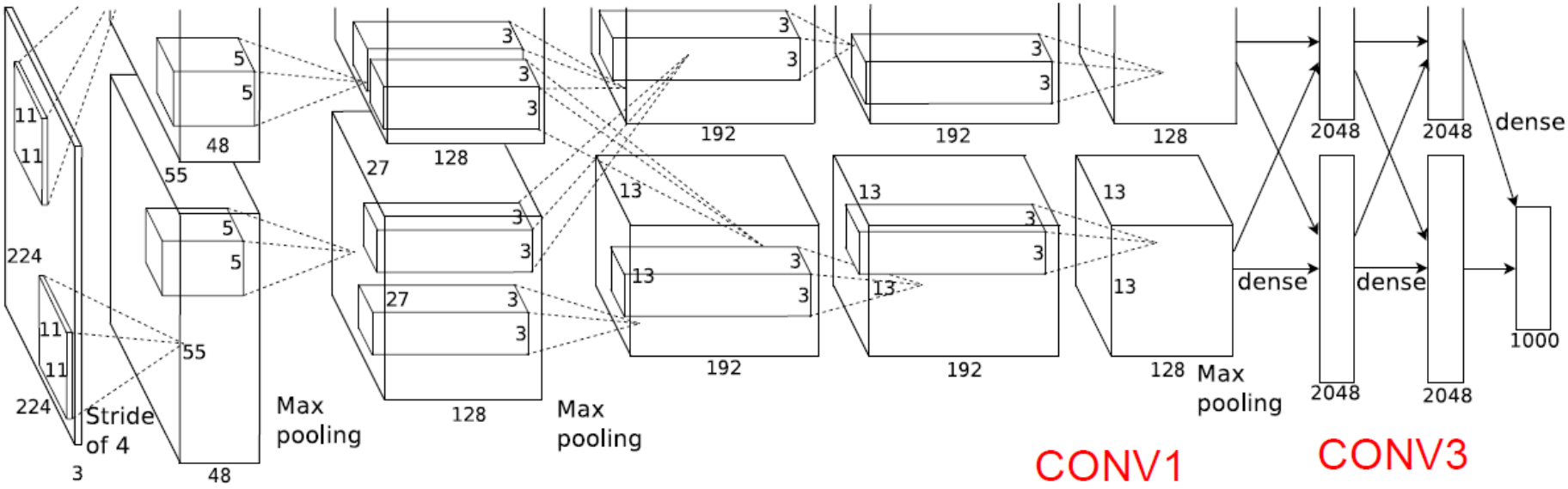


- **评价指标**
  - 精确率 Precision = TP/(TP + FP)
  - 召回率　　Recall = TP/(TP + FN)
  - PR曲线：Precision-Recall曲线
  - AP(Average Precision): PR 曲线下的面积(AUC, Area Under Curve)，是精度和召回率的函数，取值范围在0到1之间，AP越高，代表模型越好，即如果PR曲线下的面积为1，则代表模型的性能最佳
  - mAP(全类平均正确率，mean Average Precision)：将所有类别检测的平均正确率AP进行综合加权平均得到

7.2 经典图像分类网络


30
25
20
15
10
5
0
28.2
25.8
16.4
11.7
7.3
6.7
3.6
3
2.3
shallow
8 layers
8 layers
19 layers
22 layers
152 layers
152 layers
152 layers
2010
Lin et al
2011
Sanchez & Perronnin
2012
Krizhevsky et al (AlexNet)
2013
Zeiler & Fergus
2014
Simnyan & Zisserman(VGG)
2014
Szegedy et al (GoogLeNet)
2015
He et al (ResNet)
2016
Shao et al
2017
Hu et al (SENet)

1）AlexNet



- 首次利用 GPU 进行网络加速训练
- 使用了 ReLU 激活函数
- 使用了 Dropout 随机失活神经元操作，以减少过拟合

CONV1
MAX POOL1
NORM1
CONV2
MAX POOL2
NORM2
CONV3
CONV4
CONV5
Max POOL3
FC6
FC7
FC8

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky et al. 2012]

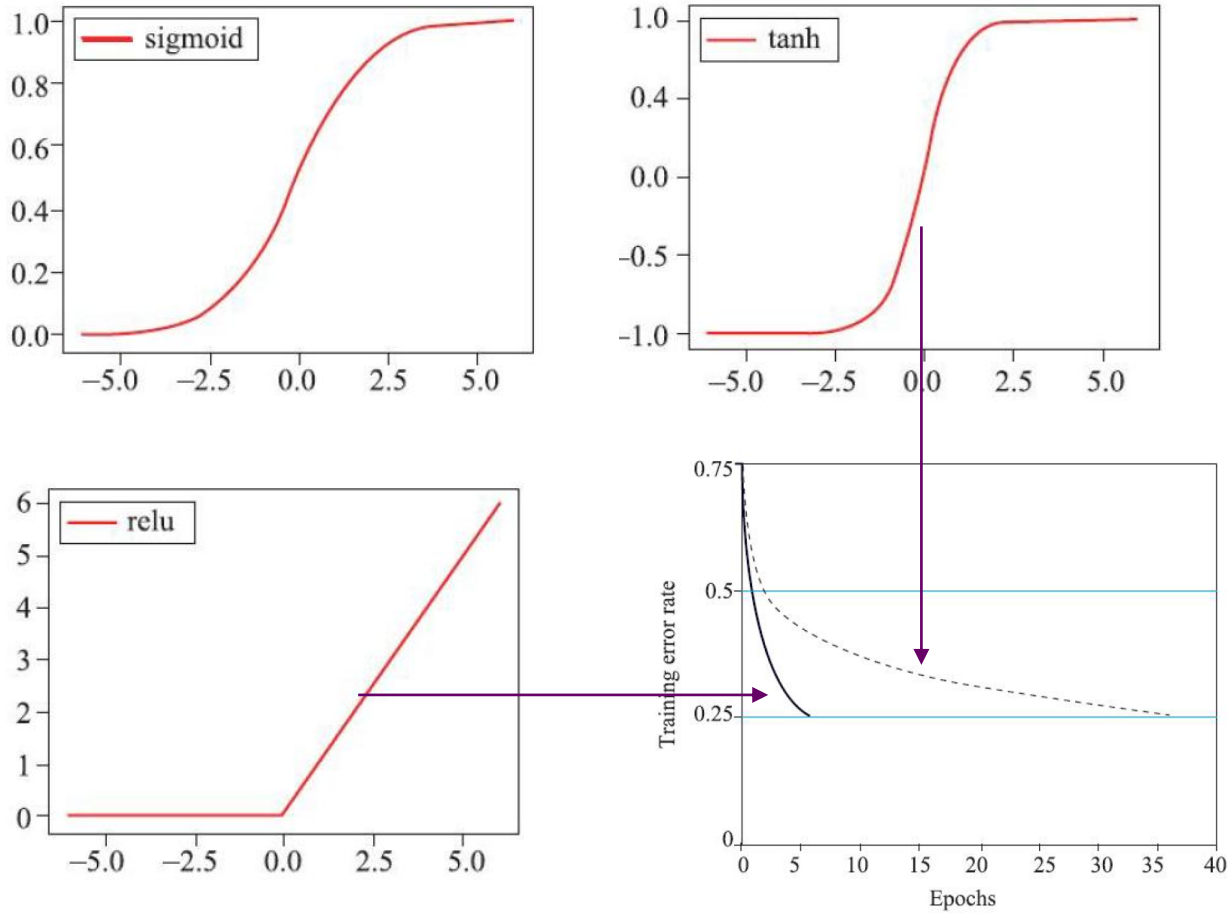
1）AlexNet

$$W' = (W - F + 2P) / S + 1$$

input	layer	kernel	kernel_num	stride	pad	output	parameters
227*227*3	CONV1	11*11*3	96	4	0	55*55*96	(11*11*3)*96 = 35K
55*55*96	MAX POOL1	3*3		2	0	27*27*96	
27*27*96	NORM1						
27*27*96	CONV2	5*5*96	256	1	2	27*27*256	(5*5*96)*256 = 614K
27*27*256	MAX POOL2	3*3		2	0	13*13*256	
13*13*256	NORM2						
13*13*256	CONV3	3*3*256	384	1	1	13*13*384	(3*3*256)*384 = 885K
13*13*384	CONV4	3*3*384	384	1	1	13*13*384	(3*3*384)*384 = 1327K
13*13*384	CONV5	3*3*384	256	1	1	13*13*256	(3*3*384)*256 = 885K
13*13*256	Max POOL3	3*3		2	0	6*6*256	
6*6*256	FC6					4096	(6*6*256)*4096 =3775W
4096	FC7					4096	4096*4096 = 1678W
4096	FC8					1000	4096* 1000 = 410W

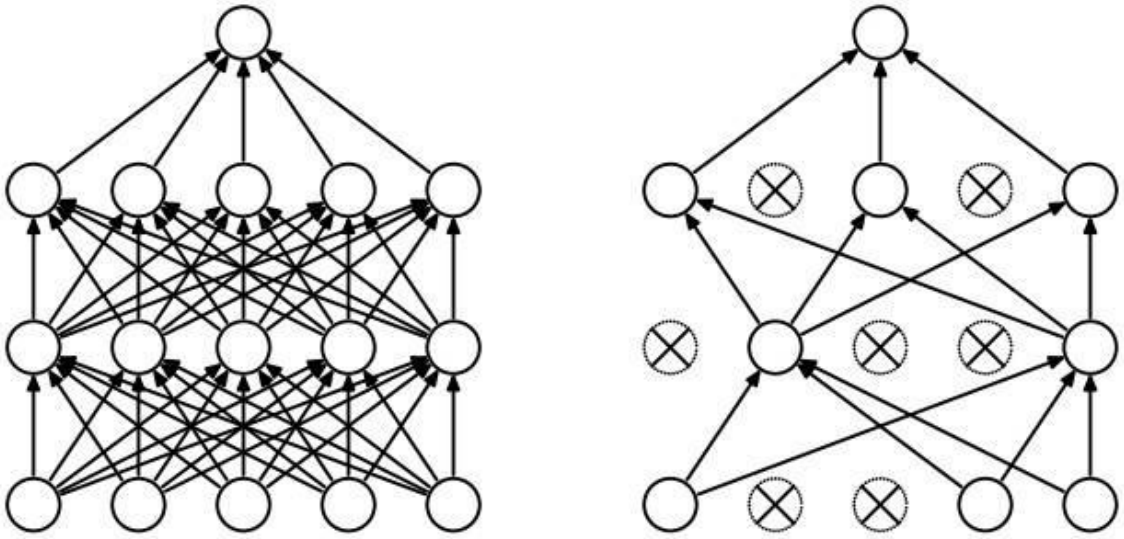
1）AlexNet

- 使用ReLu作为激活函数：从sigmoid/tanh到Relu



1）AlexNet

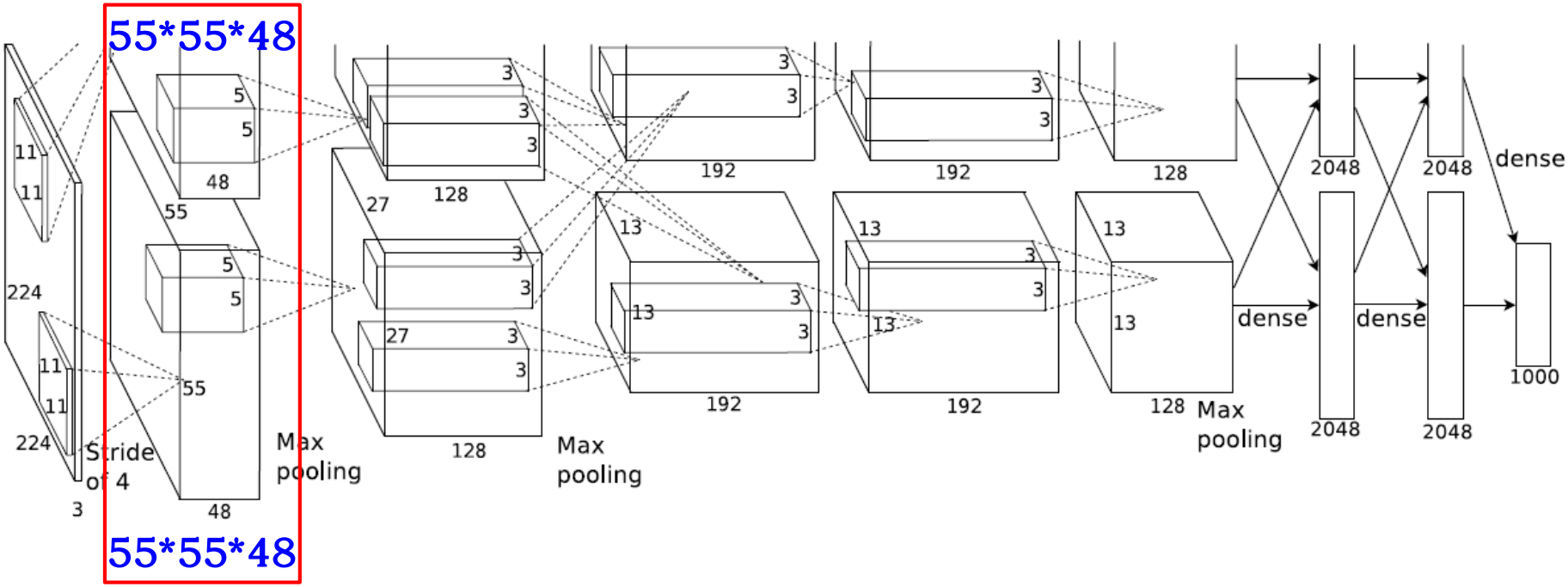
- 使用ReLu作为激活函数：从sigmoid/tanh到Relu
- 过拟合处理
  - 数据增强：随机裁剪、图像翻转、平移、缩放、旋转
  - 使用dropout：网络训练时，以一定概率随机dropout(丢弃)一些被激活的神经元，使得模型更鲁棒



(a) Standard Neural Net　　(b) After applying dropout.

1）AlexNet

- 使用ReLu作为激活函数：从sigmoid/tanh到Relu
- 过拟合处理
- 使用GPU并行



网络模型

```
class AlexNet(nn.Module):
    def __init__(self, num_classes=1000, init_weights=False):
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(# 模块打包
            nn.Conv2d(3, 48, kernel_size=11, stride=4, padding=2),  # input[3, 224, 224]  output[48, 55, 55]
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),                  # output[48, 27, 27]
            nn.Conv2d(48, 128, kernel_size=5, padding=2),           # output[128, 27, 27]
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),                  # output[128, 13, 13]
            nn.Conv2d(128, 192, kernel_size=3, padding=1),          # output[192, 13, 13]
            nn.ReLU(inplace=True),
            nn.Conv2d(192, 192, kernel_size=3, padding=1),          # output[192, 13, 13]
            nn.ReLU(inplace=True),
            nn.Conv2d(192, 128, kernel_size=3, padding=1),          # output[128, 13, 13]
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),                  # output[128, 6, 6]
        )
        self.classifier = nn.Sequential(
            nn.Dropout(p=0.5),
            nn.Linear(128 * 6 * 6, 2048),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(2048, 2048),
            nn.ReLU(inplace=True),
            nn.Linear(2048, num_classes),
        )
```

网络模型：特征提取

```
self.features = nn.Sequential(# 模块打包
    nn.Conv2d(3, 48, kernel_size=11, stride=4, padding=2),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Conv2d(48, 128, kernel_size=5, padding=2),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=3, stride=2),
    ......
```

input	layer	kernel	kernel_num	stride	pad	output
224*224*3	CONV1	11*11*3	96 /2	4	2	55*55*96
55*55*96	MAX POOL1	3*3		2	0	27*27*96
27*27*96	NORM1					
27*27*96	CONV2	5*5*96	256 /2	1	2	27*27*256
27*27*256	MAX POOL2	3*3		2	0	13*13*256
13*13*256	NORM2					
13*13*256	CONV3	3*3*256	384 /2	1	1	13*13*384
13*13*384	CONV4	3*3*384	384 /2	1	1	13*13*384
13*13*384	CONV5	3*3*384	256 /2	1	1	13*13*256

PyTorch：模块打包

```
self.features = nn.Sequential(# 模块打包
    nn.Conv2d(3, 48, kernel_size=11, stride=4, padding=2),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=3, stride=2),
```

Sequential

https://docs.pytorch.org/docs/2.11/generated/torch.nn.Sequential.html#torch.nn.Sequential

```
class torch.nn.Sequential(*args: Module)
class torch.nn.Sequential(arg: OrderedDict[str, Module])
```

A sequential container.

- 核心作用：将一系列神经网络层（比如卷积层、激活函数、池化层等）按顺序堆叠起来，形成一个完整的模块
- 像传参数一样把层传进去，PyTorch会自动分配数字索引（0, 1, 2...）
- 例如，访问第一层可以用： features[0]

网络模型：分类+前向计算

```
self.classifier = nn.Sequential(
    nn.Dropout(p=0.5),
    nn.Linear(128 * 6 * 6, 2048),
    nn.ReLU(inplace=True),
    nn.Dropout(p=0.5),
    nn.Linear(2048, 2048),
    nn.ReLU(inplace=True),
    nn.Linear(2048, num_classes),
```

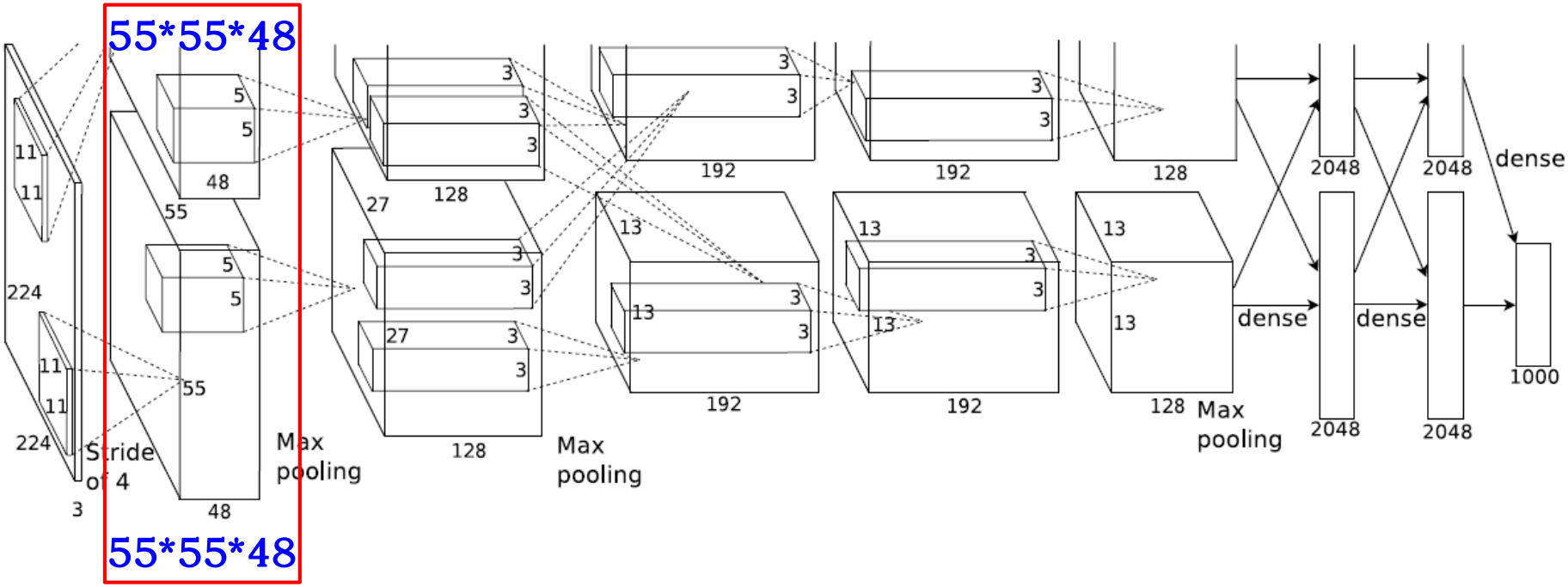
6*6*256 /2	FC6					4096
4096 /2	FC7					4096
4096 /2	FC8					1000

将多维的张量（Tensor）展平，start_dim表示从哪个维度开始展平

```
def forward(self, x):
    x = self.features(x)
    x = torch.flatten(x, start_dim=1)
    x = self.classifier(x)
    return x
```

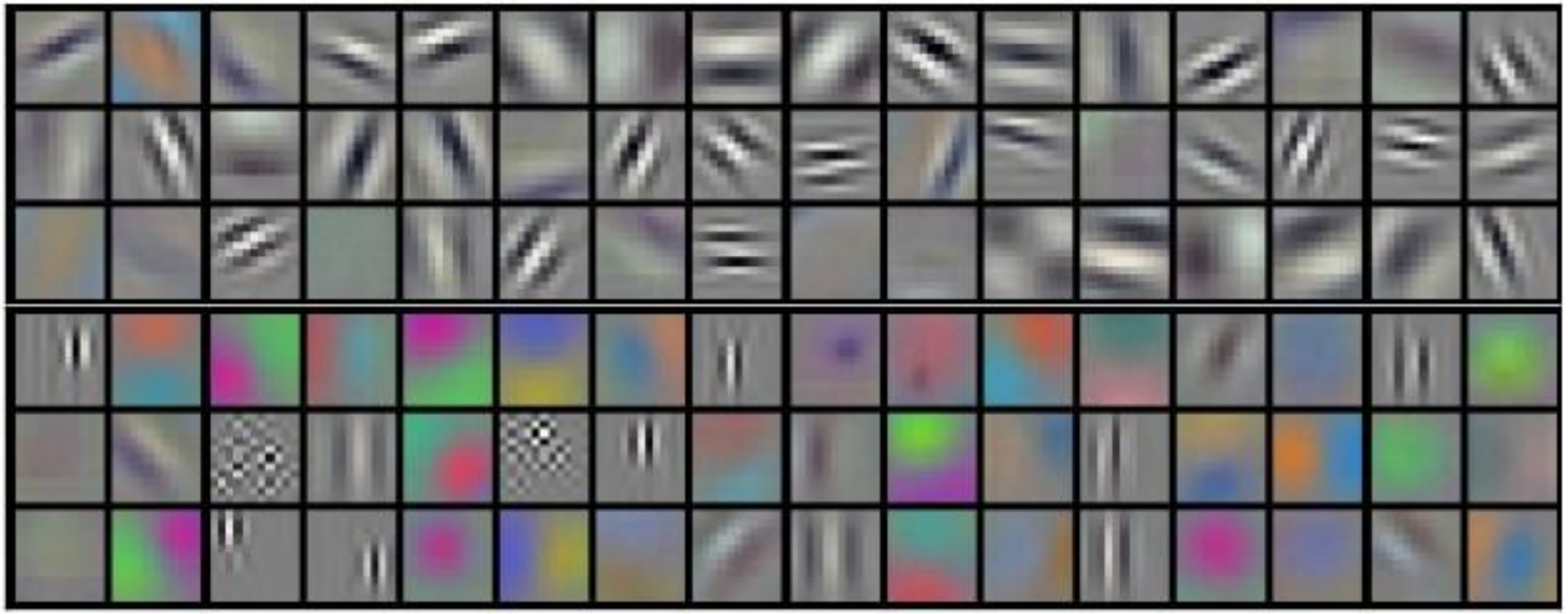
1）AlexNet

- 使用ReLu作为激活函数：从sigmoid/tanh到Relu
- 过拟合处理
- 使用GPU并行



1）AlexNet

- Conv1的96个11×11×3卷积核from AlexNet



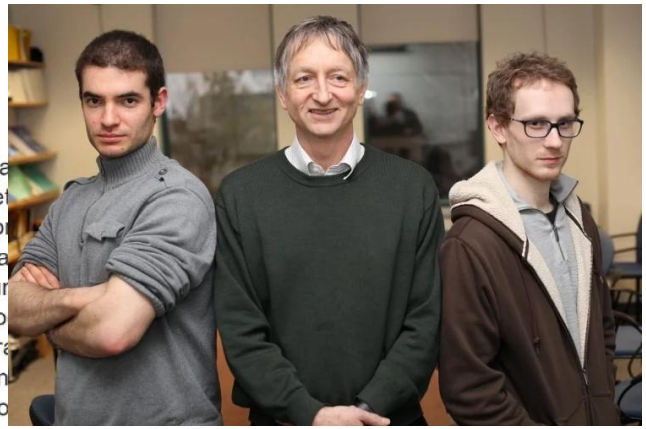
1）AlexNet

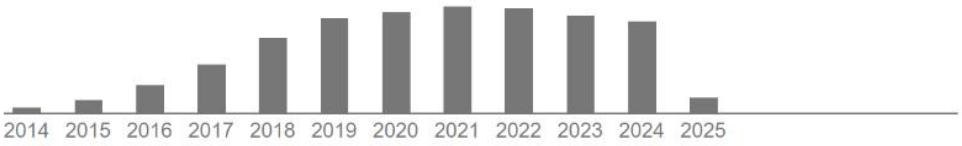
- https://github.com/computerhistory/AlexNet-Source-Code

Alex Krizhevsky

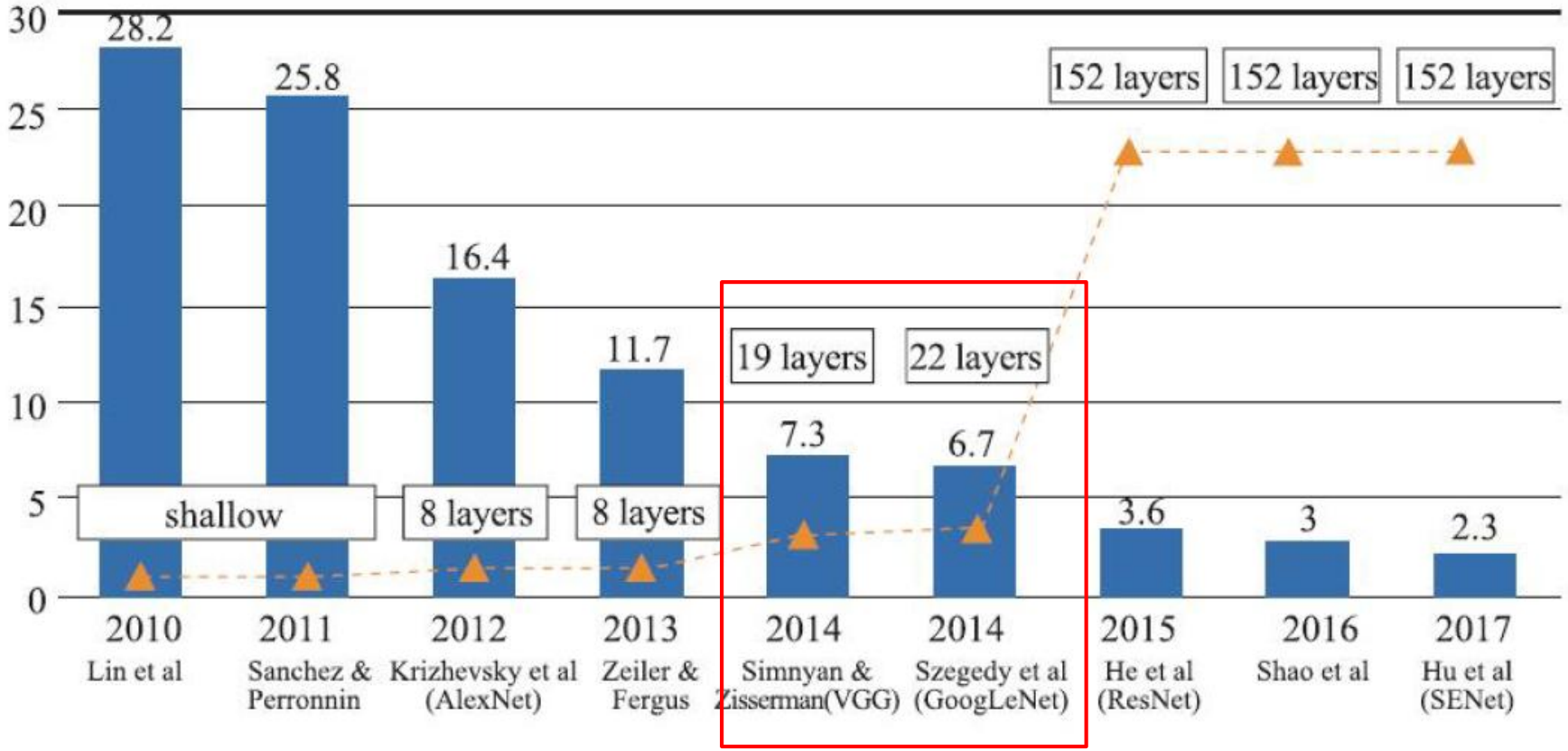
Imagenet classification with deep convolutional neural networks

Authors	Alex Krizhevsky, Ilya Sutskever, Geoffrey E Hinton
Publication date	2012
Journal	Advances in neural information processing systems
Volume	25
Description	We trained a large, deep convolutional neural network to cla resolution images in the LSVRC-2010 ImageNet training set classes. On the test data, we achieved top-1 and top-5 erro which is considerably better than the previous state-of-the-a network, which has 60 million parameters and 500,000 neur convolutional layers, some of which are followed by max-po connected layers with a final 1000-way softmax. To make tra saturating neurons and a very efficient GPU implementation reduce overfitting in the globally connected layers we emplo method that proved to be very effective.
Total citations	Cited by 173260



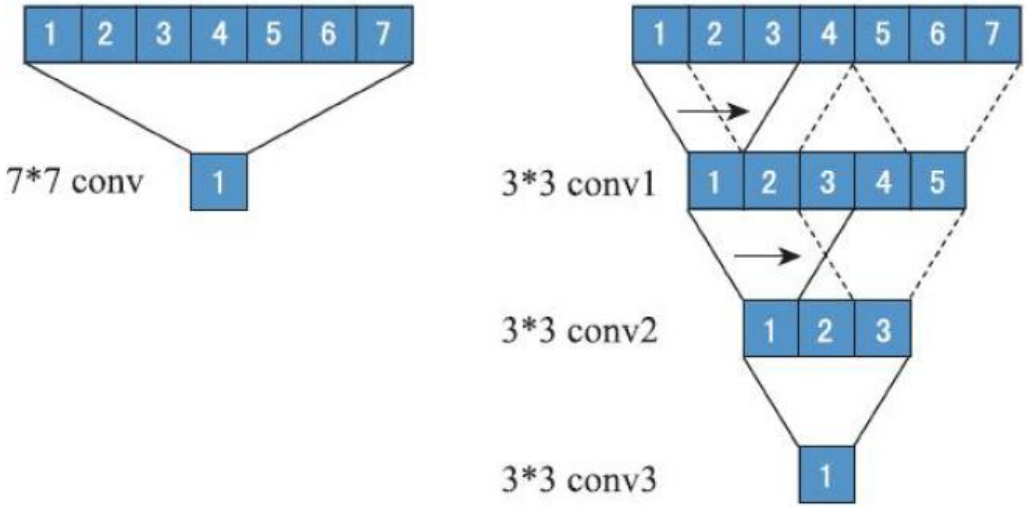


7.2 经典图像分类网络

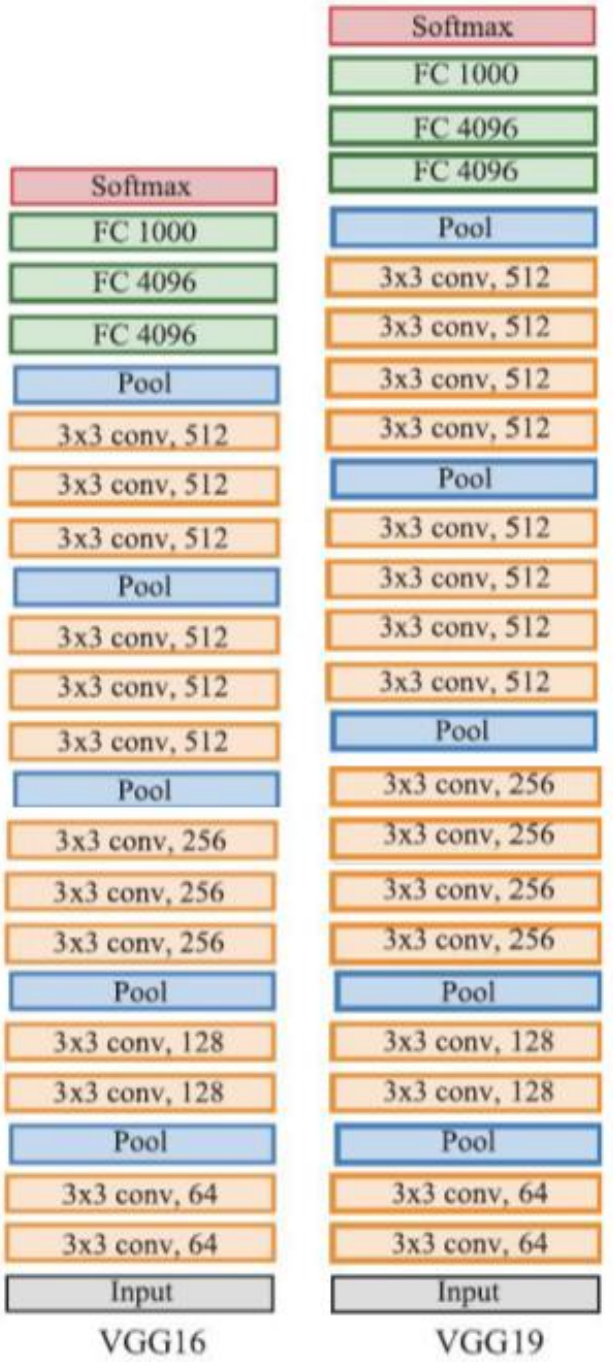

30
25
20
15
10
5
0
28.2
25.8
16.4
11.7
7.3
6.7
3.6
3
2.3
shallow
8 layers
8 layers
19 layers
22 layers
152 layers
152 layers
152 layers
2010
Lin et al
2011
Sanchez & Perronnin
2012
Krizhevsky et al (AlexNet)
2013
Zeiler & Fergus
2014
Simnyan & Zisserman(VGG)
2014
Szegedy et al (GoogLeNet)
2015
He et al (ResNet)
2016
Shao et al
2017
Hu et al (SENet)

2） VGGNet

- VGGNet：只使用3*3 kernel，3组3*3的kernel与1个7*7的kernel产生相同的效果



VGGNet-Very Deep Convolutional Networks for Large-Scale Image Recognition [Simonyan and Zisserman 2014]



VGG16 网络结构

memory:
24M * 4 bytes
≈96MB/image

params: 138M

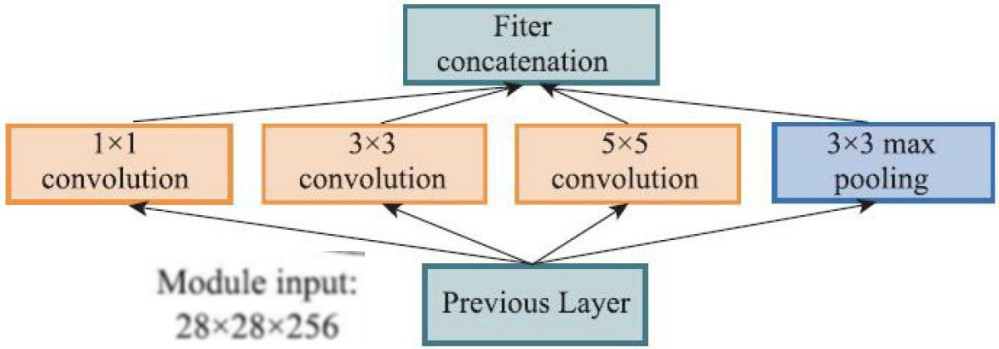
input_size	layer	memory	parameters
[224x224x3]	input	224*224*3=150K	0
[224x224x64]	CONV3-64	224*224*64=3.2M	(3*3*3)*64 = 1,728
[224x224x64]	CONV3-64	224*224*64=3.2M	(3*3*64)*64 = 36,864
[112x112x64]	POOL2	112*112*64=800K	0
[112x112x128]	CONV3-128	112*112*128=1.6M	(3*3*64)*128 = 73,728
[112x112x128]	CONV3-128	112*112*128=1.6M	(3*3*128)*128 = 147,456
[56x56x128]	POOL2	56*56*128=400K	0
[56x56x256]	CONV3-256	56*56*256=800K	(3*3*128)*256 = 294,912
[56x56x256]	CONV3-256	56*56*256=800K	(3*3*256)*256 = 589,824
[56x56x256]	CONV3-256	56*56*256=800K	(3*3*256)*256 = 589,824
[28x28x256]	POOL2	28*28*256=200K	0
[28x28x512]	CONV3-512	28*28*512=400K	(3*3*256)*512 = 1,179,648
[28x28x512]	CONV3-512	28*28*512=400K	(3*3*512)*512 = 2,359,296
[28x28x512]	CONV3-512	28*28*512=400K	(3*3*512)*512 = 2,359,296
[14x14x512]	POOL2	14*14*512=100K	0
[14x14x512]	CONV3-512	14*14*512=100K	(3*3*512)*512 = 2,359,296
[14x14x512]	CONV3-512	14*14*512=100K	(3*3*512)*512 = 2,359,296
[14x14x512]	CONV3-512	14*14*512=100K	(3*3*512)*512 = 2,359,296
[7x7x512]	POOL2	7*7*512=25K	0
[1x1x4096]	FC	4096	7*7*512*4096 = 102,760,448
[1x1x4096]	FC	4096	4096*4096 = 16,777,216
[1x1x1000]	FC	1000	4096*1000 = 4,096,000

VGG多种结构

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

3) GoogLeNet

- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构



input	layer	output_size	computation
28*28*256	1*1*128 conv	28*28*128	28*28*128*1*1*256
	3*3*192 conv	28*28*192	28*28*192*3*3*256
	5*5*96 conv	28*28*96	28*28*96*5*5*256
	3*3 pool	28*28*256	

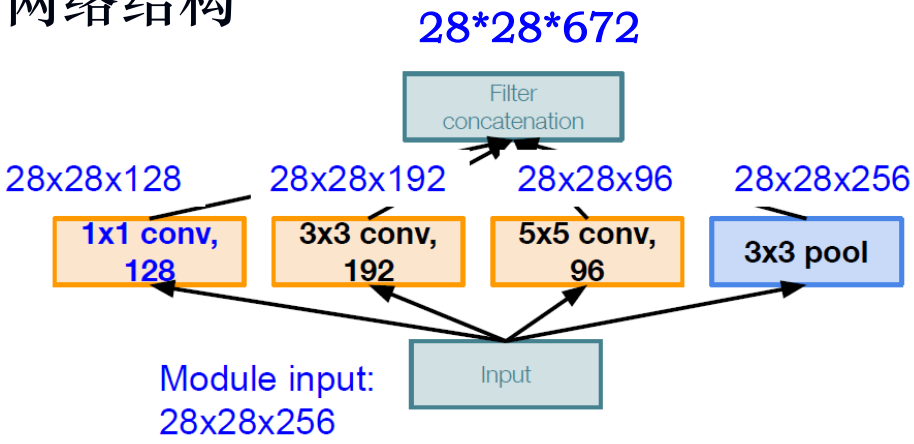
Going deeper with convolutions [Szegedy et al. 2014]

28*28*672

854M

3) GoogLeNet

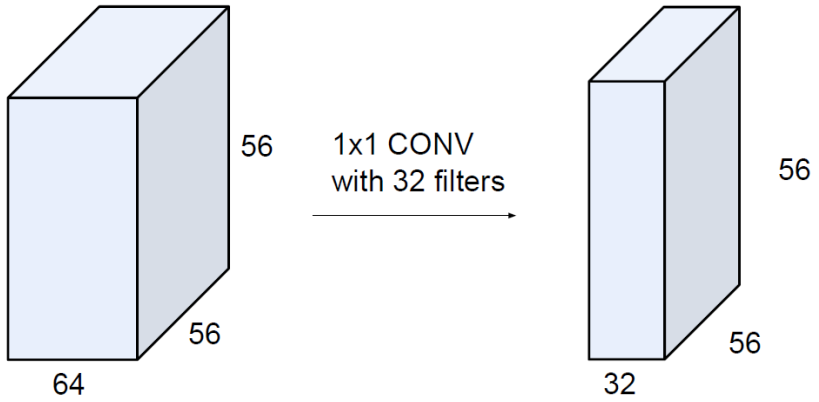
- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构



使用1x1的卷积核进行降维以及映射处理

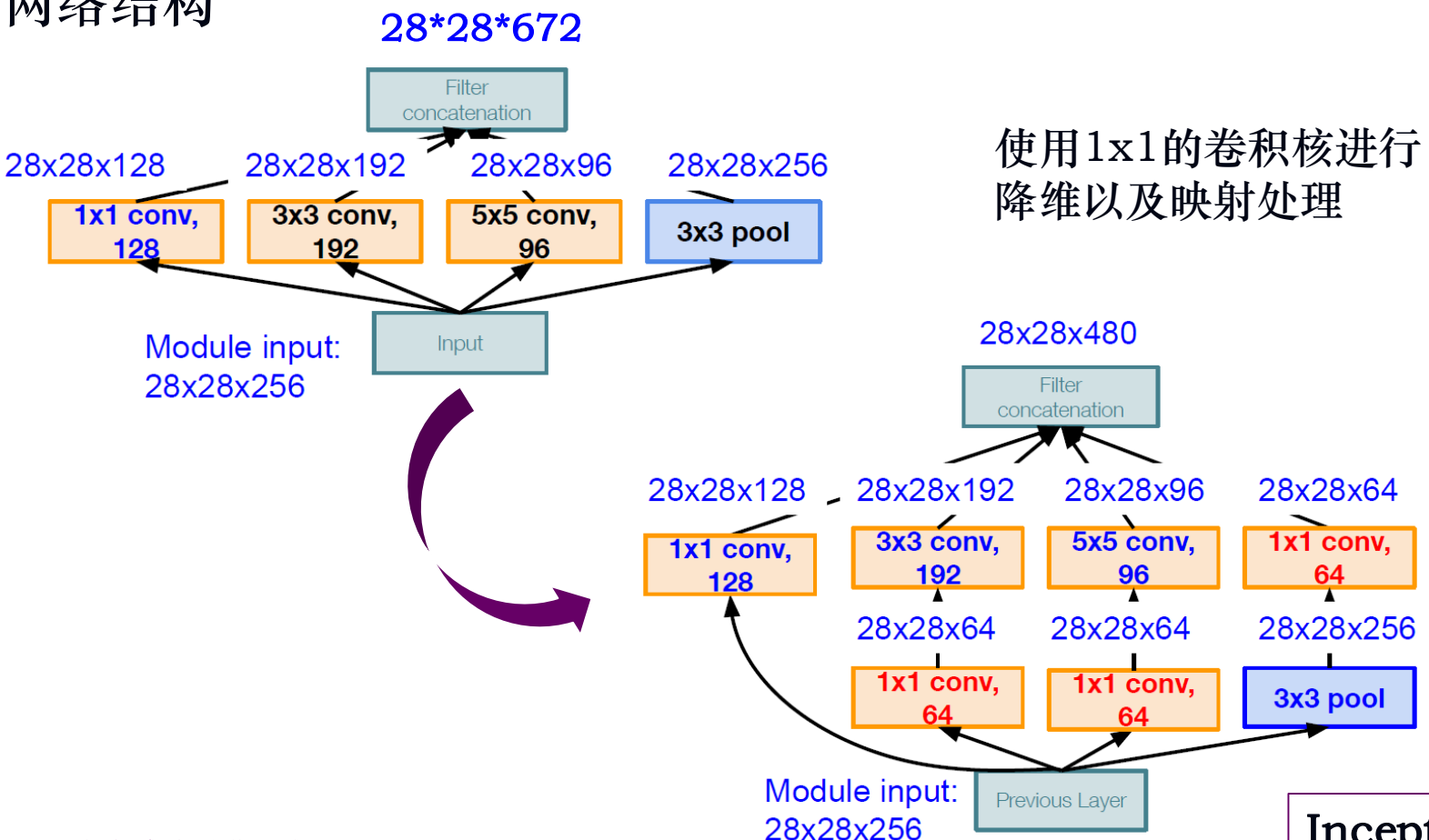
output_size	computation
28*28*128	28*28*128*1*1*256
28*28*192	28*28*192*3*3*256
28*28*96	28*28*96*5*5*256
28*28*256	

28*28*672　　854M



3) GoogLeNet

- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构



3) GoogLeNet

- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构

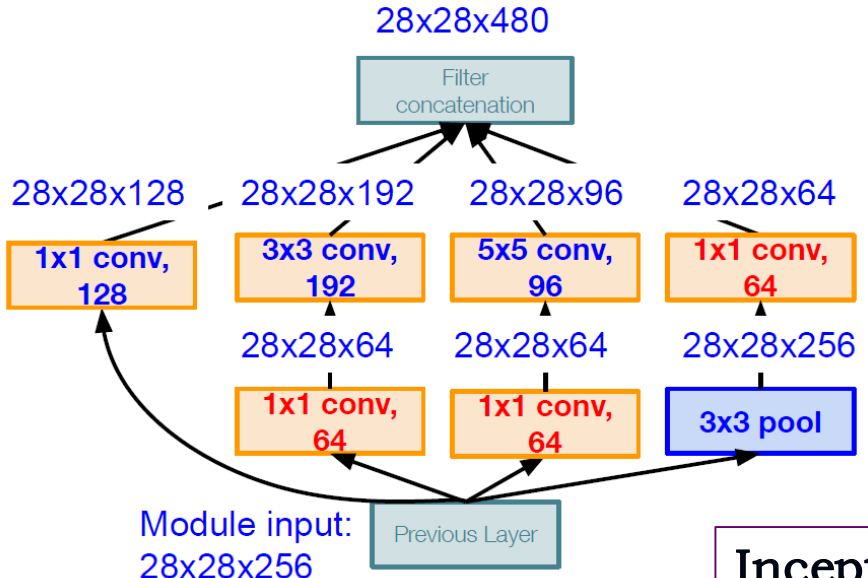
layer	computation
1*1*128 conv	28*28*128*1*1*256
3*3*192 conv	28*28*192*3*3*256
5*5*96 conv	28*28*96*5*5*256

计算量：854M

[1×1 conv, 64] 28×28×64×1×1×256
[1×1 conv, 64] 28×28×64×1×1×256
[1×1 conv, 128] 28×28×128×1×1×256
[3×3 conv, 192] 28×28×192×3×3×64
[5×5 conv, 96] 28×28×96×5×5×64
[1×1 conv, 64] 28×28×64×1×1×256

358M

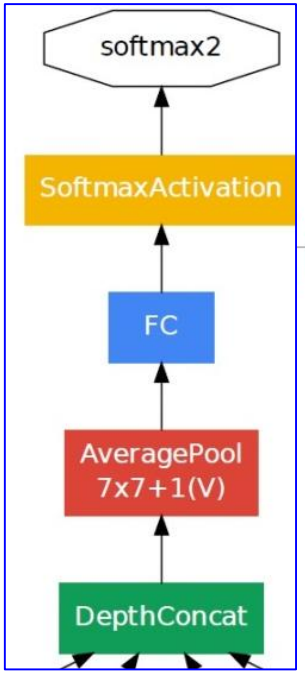
params: 5M
AlexNet的1/12
VGG-16的1/27

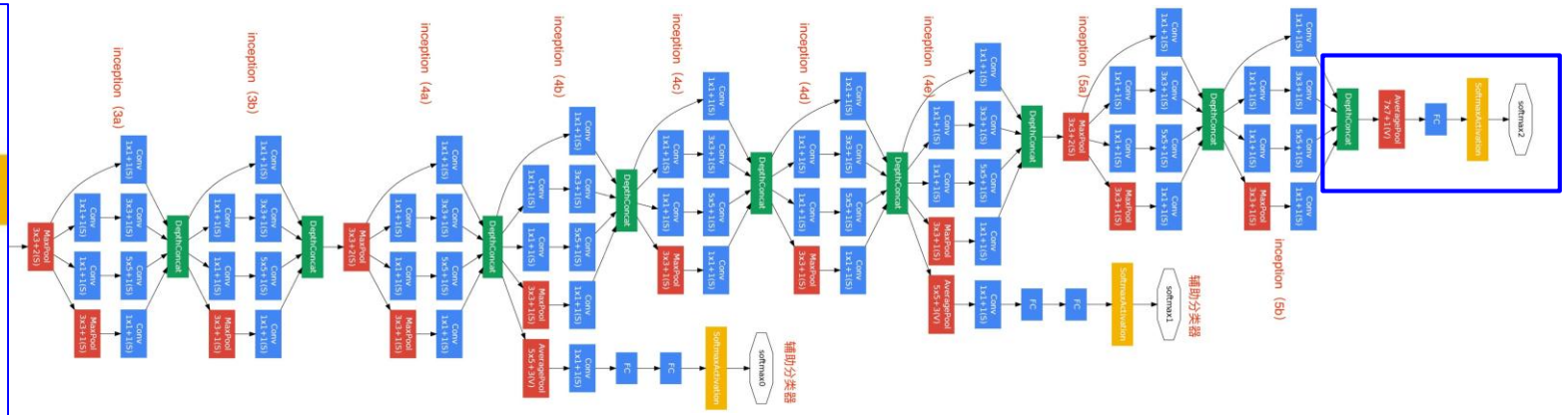


Inception v1

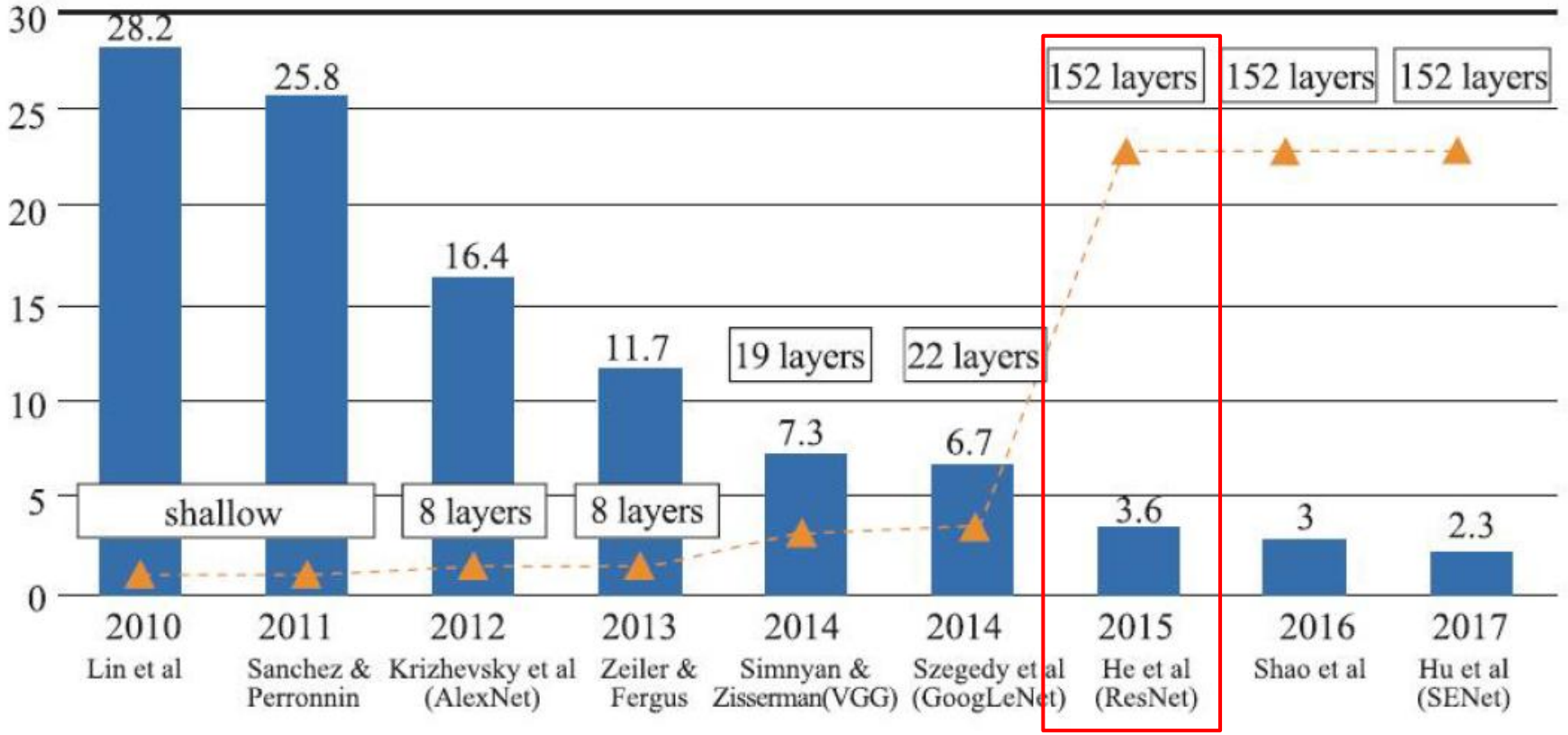
3) GoogLeNet

- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构
- 传统结构+Inception结构+分类器（使用平均池化层，只有一个全连接层）



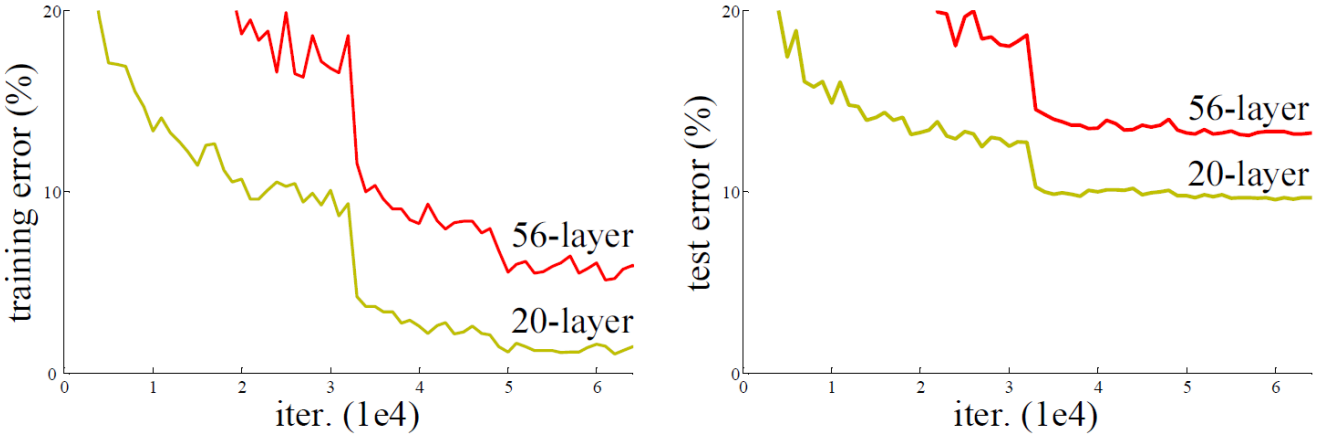


7.2 经典图像分类网络


30
25
20
15
10
5
0
28.2
25.8
16.4
11.7
7.3
6.7
3.6
3
2.3
shallow
8 layers
8 layers
19 layers
22 layers
152 layers
152 layers
152 layers
2010
Lin et al
2011
Sanchez & Perronnin
2012
Krizhevsky et al (AlexNet)
2013
Zeiler & Fergus
2014
Simnyan & Zisserman(VGG)
2014
Szegedy et al (GoogLeNet)
2015
He et al (ResNet)
2016
Shao et al
2017
Hu et al (SENet)

4）ResNet

- *Is learning better networks as easy as stacking more layers?*
- 网络退化（degradation）

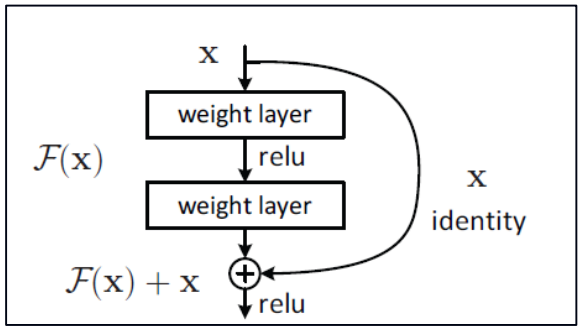


- 恒等映射(Identity Mapping)
  - 把低层的特征传到高层，效果应该至少不比浅层的网络效果差
  - 在网络高层和低层之间加入恒等映射来达到此效果

Deep Residual Learning for Image Recognition [He et al. 2015]

4）ResNet

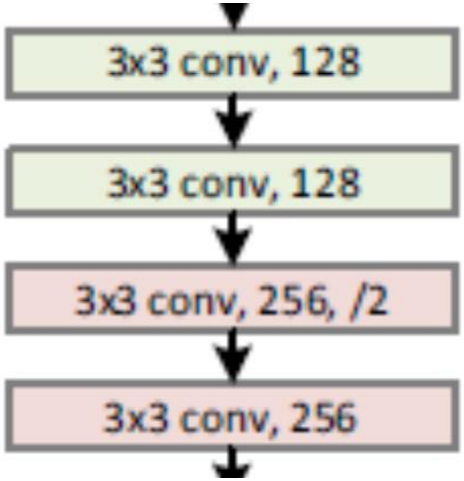
- **残差网络**
  - $F(x)+x$可以通过在前馈网络中加入“快捷连接”(shortcut connections)来实现

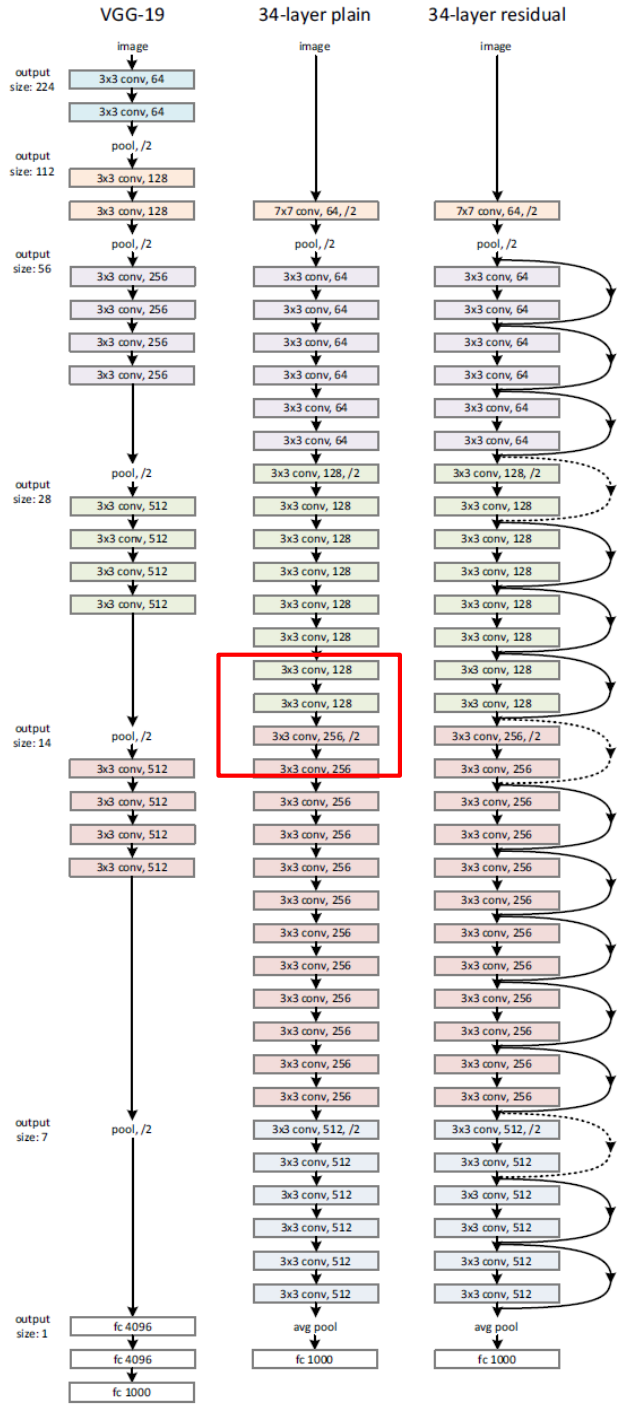



```
def forward(self, x):
    identity = x    # shortcut分支

    # 主分支
    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)
    out = self.conv2(out)
    out = self.bn2(out)

    # 注意此处没有激活，加上shortcut后，再进行激活
    out += identity
    out = self.relu(out)

    return out
```

4）ResNet

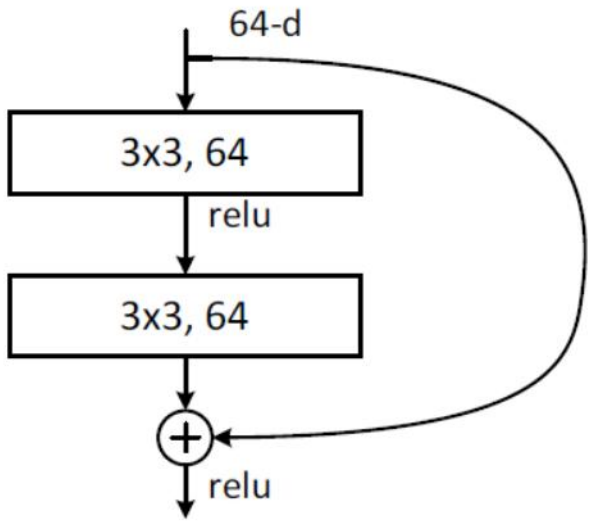
- 网络结构(Plain Network)
  - 对于相同尺寸的输出特征图，每层必须含有相同数量的卷积核
  - 如果特征图的尺寸减半，则卷积核数量翻倍，以保持每层的时间复杂度
  - 一个全连接层

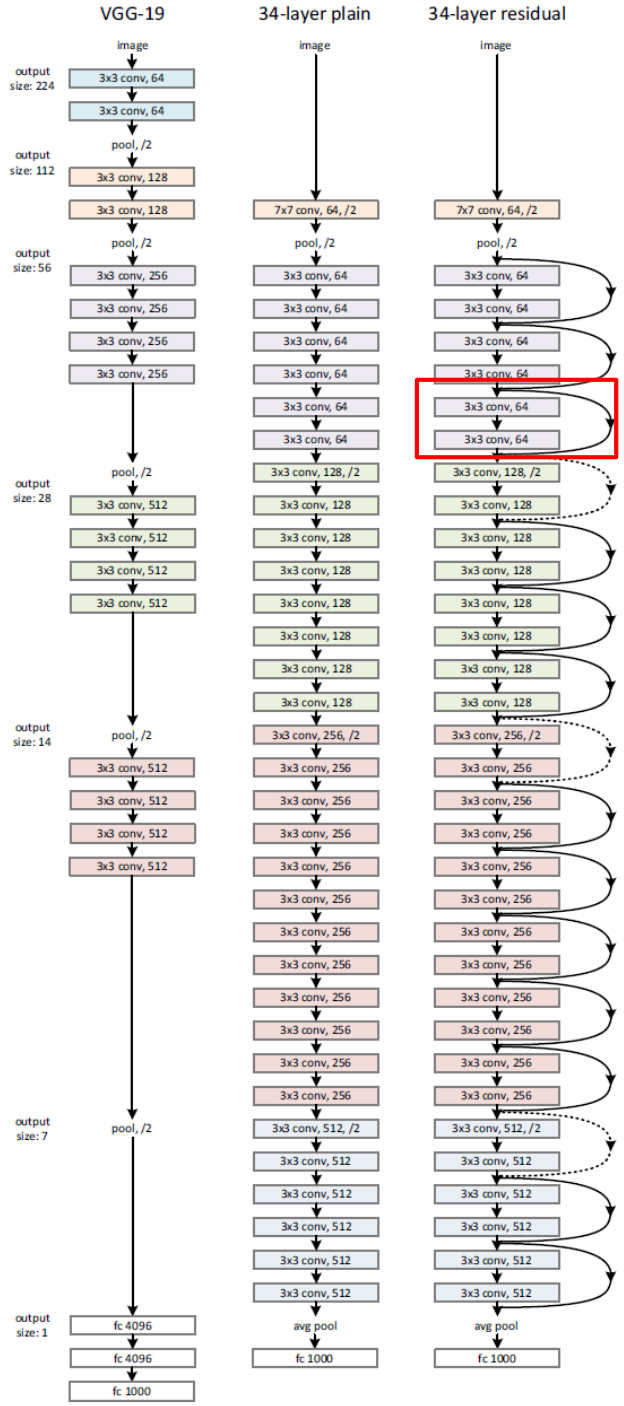




4）ResNet

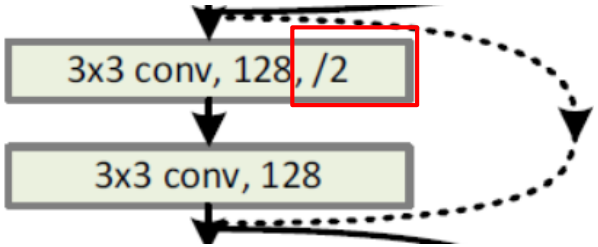
- 网络结构(ResNet18/34)
  - 加入shortcut connections，形成网络的残差版本，每个残差模块有2个卷积层(图中实线)

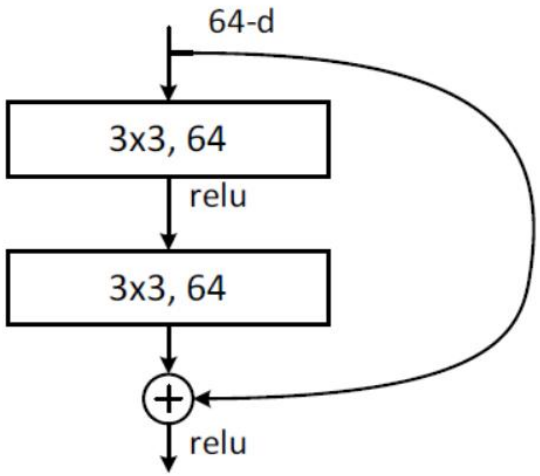


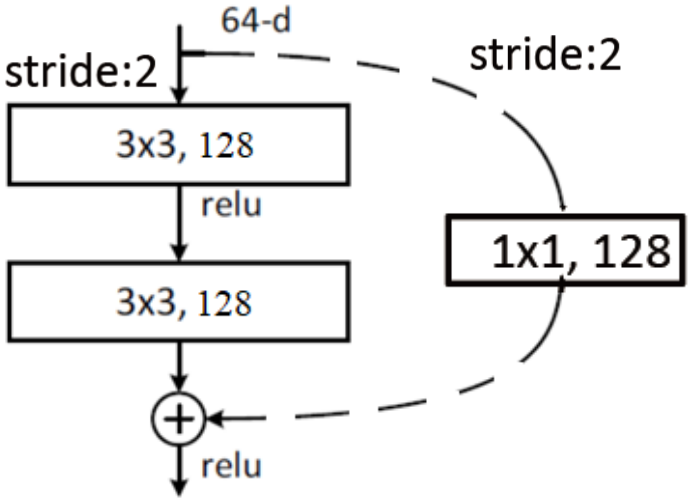


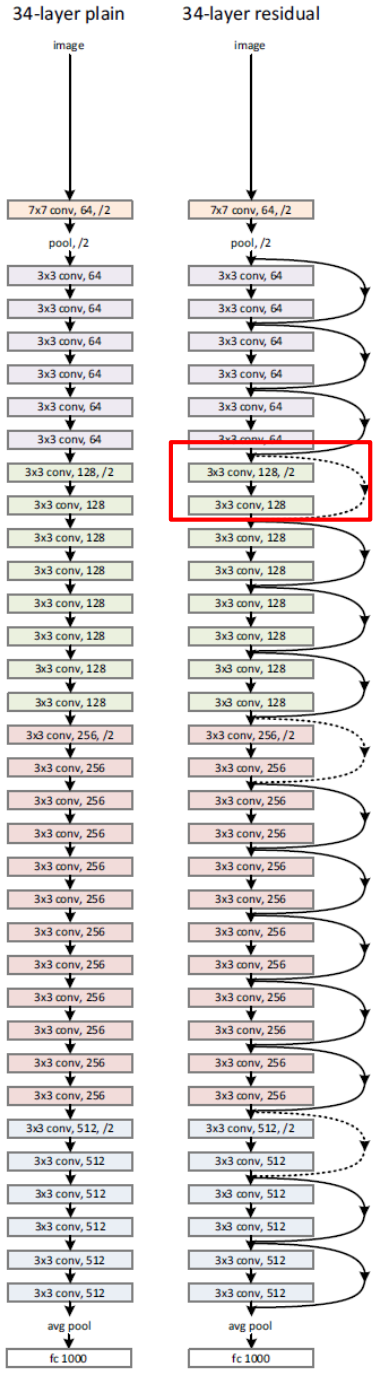
4）ResNet

- 网络结构(ResNet18/34)
  - 输入输出尺寸不同的处理：补零或使用1*1的卷积（图中虚线）



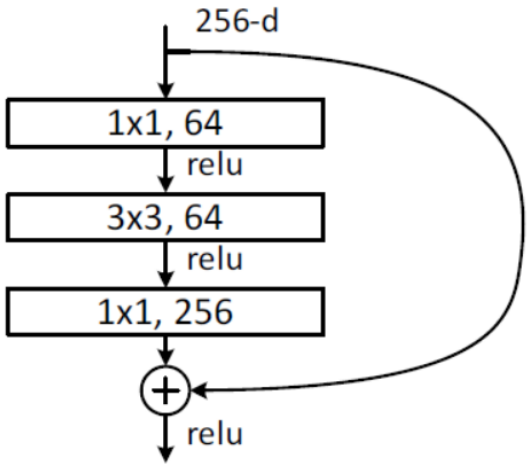


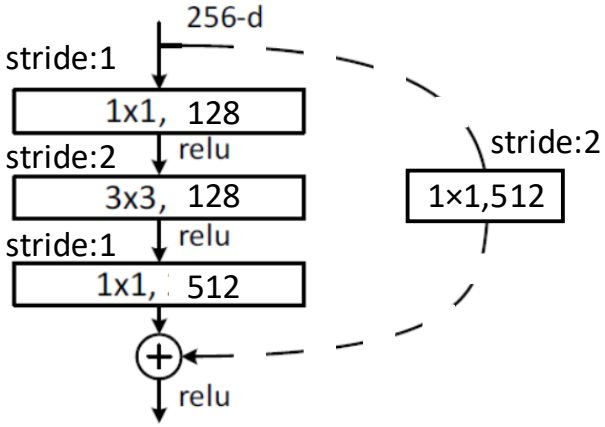




4）ResNet

- 网络结构(ResNet50/101/152)





layer name	output size	152-layer
conv1	112×112	
conv2_x	56×56	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix} \times 3$
	1×1	
FLOPs		11.3×10^9

4）ResNet

- 网络结构（5种）

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

4）ResNet：网络架构

- 网络结构（5种）

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	[3×3, 64; 3×3, 64] ×2	[3×3, 64; 3×3, 64] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3
conv3_x	28×28	[3×3, 128; 3×3, 128] ×2	[3×3, 128; 3×3, 128] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×8
conv4_x	14×14	[3×3, 256; 3×3, 256] ×2	[3×3, 256; 3×3, 256] ×6	[1×1, 256; 3×3, 256; 1×1, 1024] ×6	[1×1, 256; 3×3, 256; 1×1, 1024] ×23	[1×1, 256; 3×3, 256; 1×1, 1024] ×36
conv5_x	7×7	[3×3, 512; 3×3, 512] ×2	[3×3, 512; 3×3, 512] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

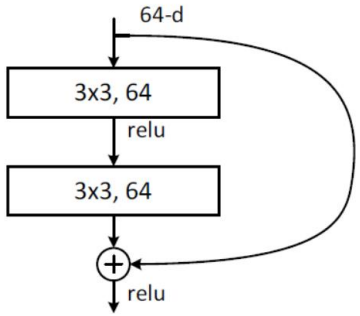
基础模块：BasicBlock

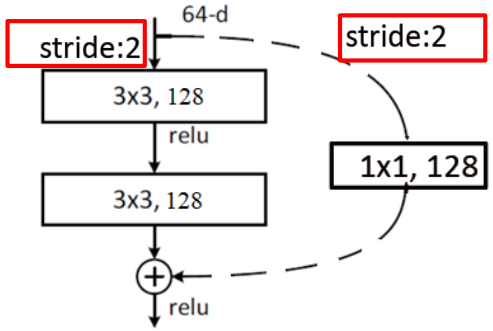
基础模块：Bottleneck

4）ResNet：网络架构

model.py

```
def forward(self, x):
    identity = x
    if self.downsample is not None:
        identity = self.downsample(x)

    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)

    out = self.conv2(out)
    out = self.bn2(out)

    out += identity
    out = self.relu(out)

    return out
```





残差结构(ResNet34)

```
if stride != 1 or self.in_channel != channel * block.expansion:
    downsample = nn.Sequential(
        nn.Conv2d(self.in_channel, channel * block.expansion,
                  kernel_size=1, stride=stride, bias=False),
        nn.BatchNorm2d(channel * block.expansion))
```

编程讲解说明

- https://www.bilibili.com/video/BV14E411H7Uw/?spm_id_from=333.1387.collection.video_card.click&vd_source=9b166dc770f19e6acbdebfdaa1dd8810

1.1 卷积神经网络基础

15万 2020-2-25

1.2 卷积神经网络基础补充

8.1万 2020-2-25

2.1 pytorch官方demo(Lenet)

15.9万 2020-2-25

2.2 tensorflow2官方demo

4.5万 2020-2-25

3.1 AlexNet网络结构详解与花分类数据集下载

11.3万 2020-2-25

3.2 使用pytorch搭建AlexNet并训练花分类数据集

14万 2020-2-25

3.3 使用tensorflow2搭建Alexnet

3.5万 2020-2-25

4.1 VGG网络详解及感受野的计算

7.7万 2020-2-25

4.2 使用pytorch搭建VGG网络

8.2万 2020-2-25

4.3 使用tensorflow搭建VGG网络

2.6万 2020-2-25

5.1 GoogLeNet网络详解

5.7万 2020-2-25

5.2 使用pytorch搭建GoogLeNet网络

4.7万 2020-2-25

5.3 使用tensorflow搭建GoogLeNet网络

1.5万 2020-2-25

6.1 ResNet网络结构，BN以及迁移学习详解

17.7万 2020-2-27

6.1.2 ResNeXt网络结构

6万 2021-2-5

6.2 使用pytorch搭建ResNet并基于迁移学习训练

15.9万 2020-2-29

6.2.2 使用pytorch搭建ResNeXt并基于迁移学习训练

3.3万 2021-2-6

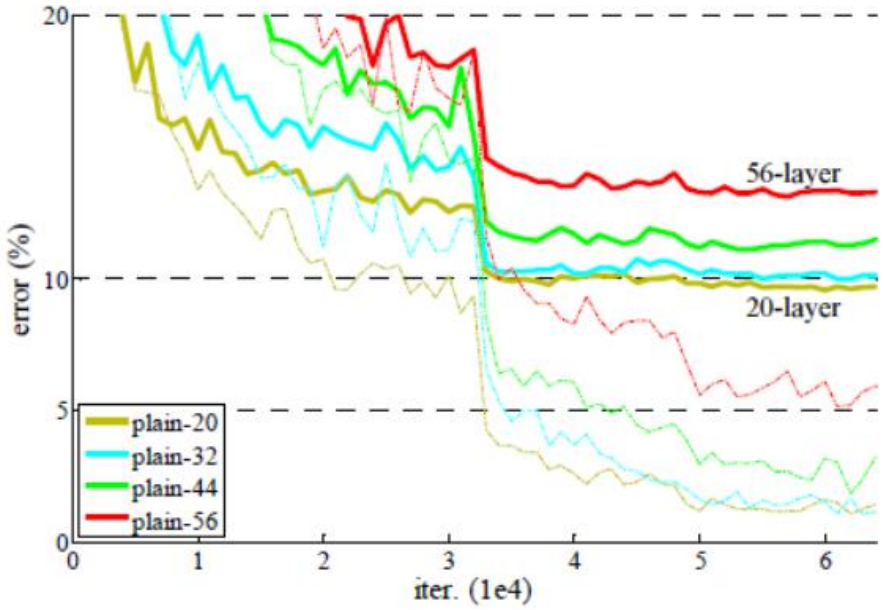
6.3 使用tensorflow搭建ResNet网络并基于迁移学习的方法进行训练

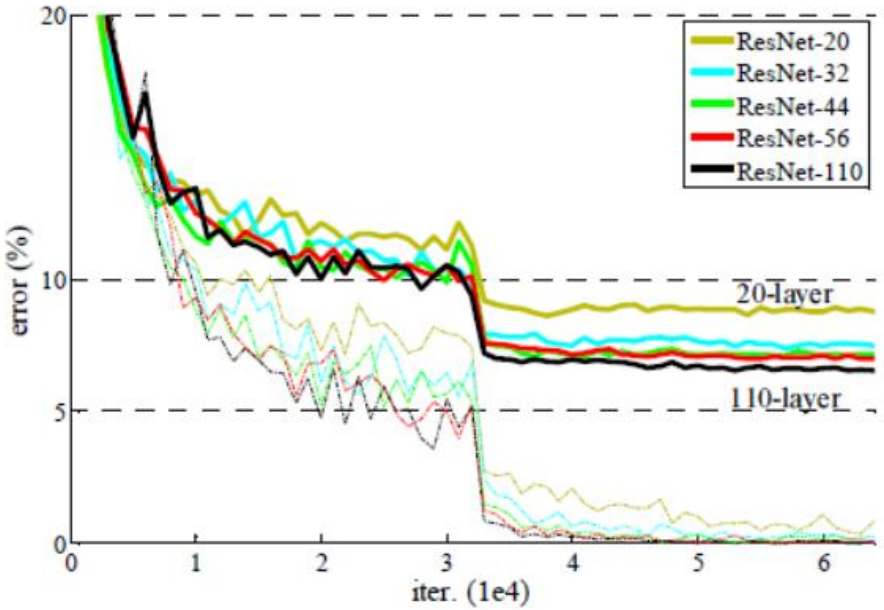
3.2万 2020-3-3

4）ResNet

- **残差网络效果**
  - 训练深层网络不会出现退化
  - 深层网络训练误差更小

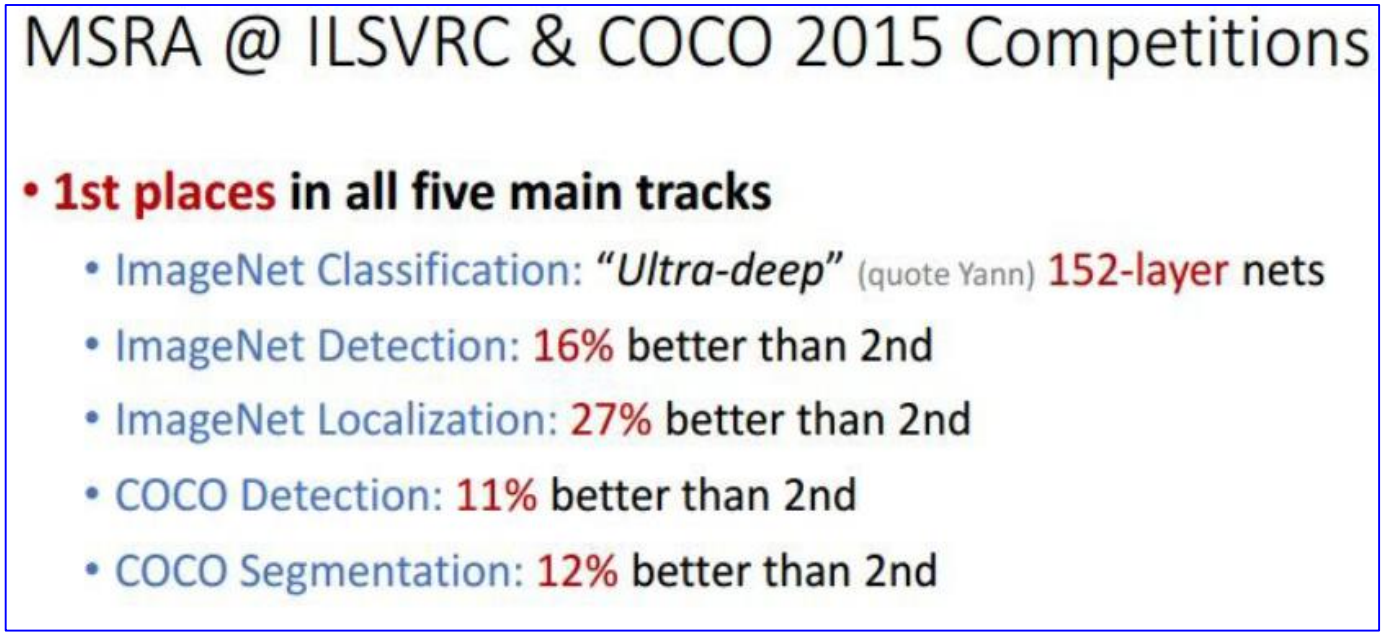
	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	**25.03**





4）ResNet

- **残差网络效果**
  - 训练深层网络不会出现退化
  - 深层网络训练误差更小
  - 囊括了ILSVRC'15和COCO'15的所有分类和检测比赛冠军!



4）ResNet

- 21世纪被引量最多的论文
- https://www.nature.com/articles/d41586-025-01125-9

NEWS FEATURE | 15 April 2025 | Clarification 22 April 2025

Exclusive: the most-cited papers of the twenty-first century

A *Nature* analysis reveals the 25 highest-cited papers published this century and explores why they are breaking records.

By Helen Pearson, Heidi Ledford, Matthew Hutson & Richard Van Noorden

TOP TEN MOST-CITED WORKS OF THE TWENTY-FIRST CENTURY

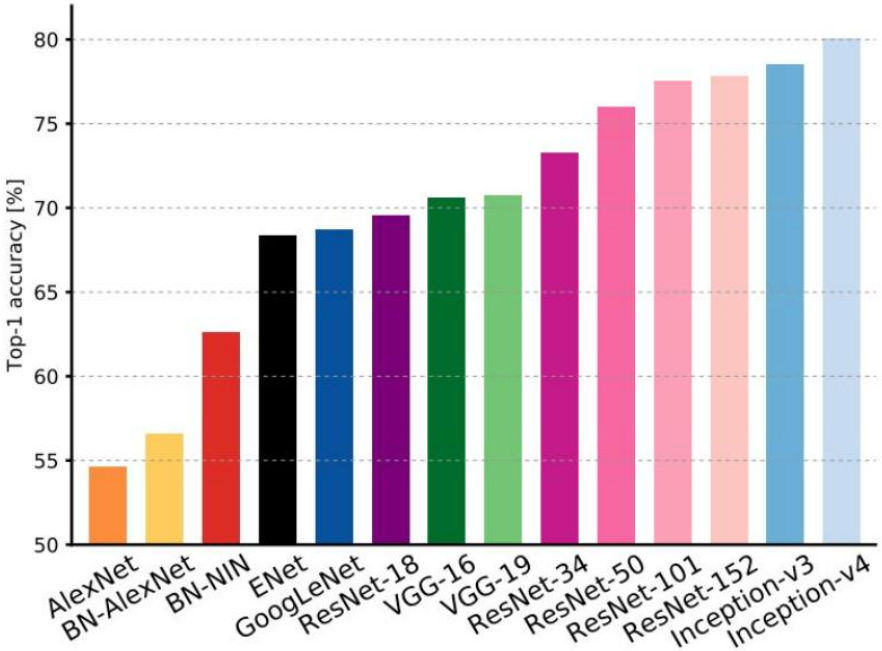
Rank (median)	Citation	Times cited (range across databases)
1	Deep residual learning for image recognition (2016, preprint 2015)	103,756–254,074
2	Analysis of relative gene expression data using real-time quantitative PCR and the $2^{-\Delta\Delta C_T}$ method (2001)	149,953–185,480
3	Using thematic analysis in psychology (2006)	100,327–230,391
4	*Diagnostic and Statistical Manual of Mental Disorders, DSM-5* (2013)	98,312–367,800
5	A short history of SHELX (2007)	76,523–99,470
6	Random forests (2001)	31,809–146,508
7	Attention is all you need (2017)	56,201–150,832
8	ImageNet classification with deep convolutional neural networks (2017)	46,860–137,997
9	Global cancer statistics 2020: GLOBOCAN estimates of incidence and mortality worldwide for 36 cancers in 185 countries (2020)	75,634–99,390
10	Global cancer statistics 2018: GLOBOCAN estimates of incidence and mortality worldwide for 36 cancers in 185 countries (2016)	66,844–93,433

Source databases: Web of Science, Scopus, OpenAlex, Dimensions, Google Scholar.

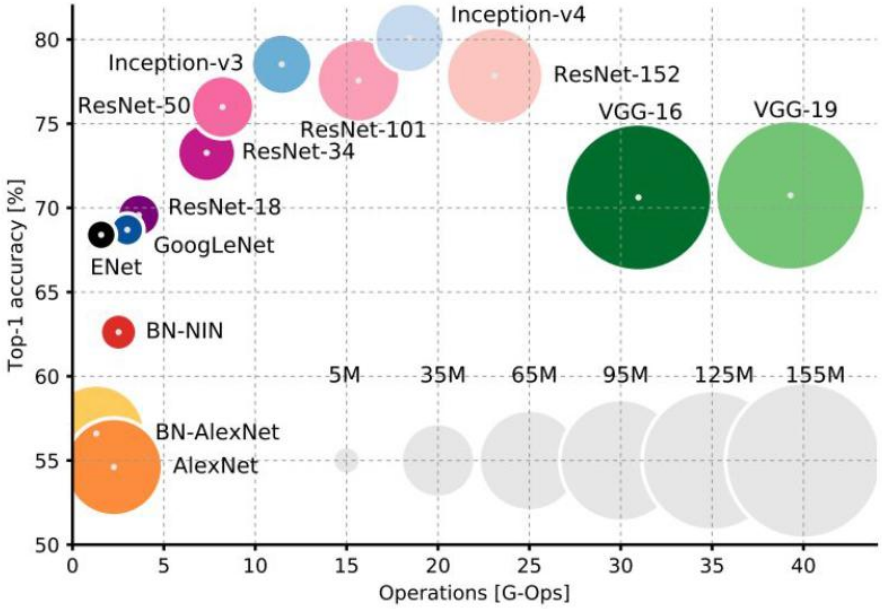
For a list of the top 25 most-cited works, see Supplementary information (go.nature.com/4tsggtd).

总结

- AlexNet ➡ 使用ReLu激活函数、引入dropout处理过拟合
- VGGNet ➡ 小卷积核（3*3）的使用
- GoogLeNet ➡ 引入Inception结构
- ResNet ➡ 引入残差学习



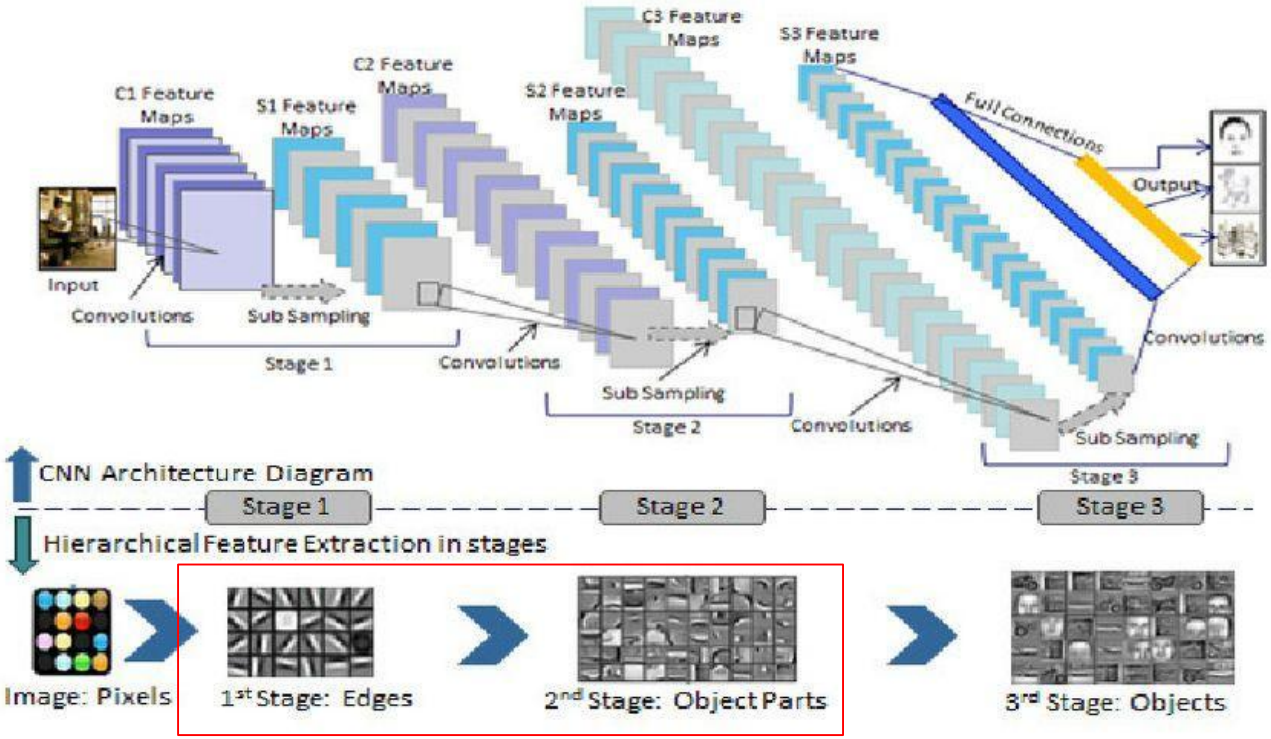
a）网络结构可达到的最高精度



b）网络结构精度vs计算量vs内存消耗

7.3 迁移学习(Transfer learning)

- 利用迁移学习，把已训练好的模型参数迁移到新的模型来帮助新模型训练，从而降低训练成本



7.3 迁移学习(Transfer learning)

- 利用迁移学习，把已训练好的模型参数迁移到新的模型来帮助新模型训练，从而降低训练成本
- 迁移学习的优势：
  - 缩短训练时间：利用预训练模型作为起点，大大减少了训练时间
  - 数据需求更少：即使在目标任务数据量很小的情况下，也能取得很好的效果，解决了“数据稀缺”和“冷启动”难题
  - 提升模型性能：预训练模型已经学到了通用的特征，通常能获得比从头训练更好的准确率和泛化能力
  - 成本效益高：复用现有模型，降低了计算和存储成本

7.3 迁移学习(Transfer learning)

- 迁移学习方式：预训练 + 微调

举例：花分类迁移学习

- 加载在 ImageNet 上训练好的ResNet34权重
- 将 ResNet34的所有特征提取层参数冻结
- 把最后的分类层换成适合“花卉识别（5分类）”的新层
- 只训练这个新换的分类层，利用旧模型提取特征

```
"0": "daisy",
"1": "dandelion",
"2": "roses",
"3": "sunflowers",
"4": "tulips"
```

举例：花分类迁移学习

train.py

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print("using {} device.".format(device))
```

选择计算设备

```
data_transform = {
    "train": transforms.Compose([transforms.RandomResizedCrop(224),
                                 transforms.RandomHorizontalFlip(),
                                 transforms.ToTensor(),
                                 transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])]),
    "val": transforms.Compose([transforms.Resize(256),
                               transforms.CenterCrop(224),
                               transforms.ToTensor(),
                               transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])}
```

图像预处理与增强

```
data_root = os.path.abspath(os.path.join(os.getcwd(), "../../"))  # get data root path
image_path = os.path.join(data_root, "dataset", "flower_data")  # flower data set path
assert os.path.exists(image_path), "{} path does not exist.".format(image_path)
```

获取数据路径

举例：花分类迁移学习

train.py


```
train_dataset = datasets.ImageFolder(root=os.path.join(image_path, "train"),
                                     transform=data_transform["train"])
train_num = len(train_dataset)
```

加载数据集

```
# {'daisy':0, 'dandelion':1, 'roses':2, 'sunflower':3, 'tulips':4}
flower_list = train_dataset.class_to_idx
cla_dict = dict((val, key) for key, val in flower_list.items())
# write dict into json file
json_str = json.dumps(cla_dict, indent=4)
with open('class_indices.json', 'w') as json_file:
    json_file.write(json_str)
```

生成并保存类别索引字典

```
batch_size = 16
nw = min([os.cpu_count(), batch_size if batch_size > 1 else 0, 8])  # number of workers
print('Using {} dataloader workers every process'.format(nw))

train_loader = torch.utils.data.DataLoader(train_dataset,
                                           batch_size=batch_size, shuffle=True,
                                           num_workers=nw)
```

举例：花分类迁移学习

train.py

```
① net = resnet34()
  model_weight_path = "./resnet34-pre.pth"
② for param in net.parameters():
      param.requires_grad = False

  # change fc layer structure
③ in_channel = net.fc.in_features
  net.fc = nn.Linear(in_channel, 5)
  net.to(device)

  # define loss function
  loss_function = nn.CrossEntropyLoss()

  # construct an optimizer
④ params = [p for p in net.parameters() if p.requires_grad]
  optimizer = optim.Adam(params, lr=0.0001)
```

1. 通过 load_state_dict获取预训练权重
2. 冻结ResNet34 所有层的参数，预训练模型作为一个强大的“通用特征提取器”
3. 重新定义一个输出为 5 个神经元的新层，net.fc是ResNet的最后一层（全连接层）

注意：新替换的net.fc 层参数随机初始化，且requires_grad 默认为 True（可训练的）

4. 仅微调（Fine-tune）新定义的net.fc层的参数

举例：花分类迁移学习

train.py

```
epochs = 3
best_acc = 0.0
save_path = './resNet34.pth'
train_steps = len(train_loader)
for epoch in range(epochs):
    # train
    net.train()
    running_loss = 0.0
    train_bar = tqdm(train_loader, file=sys.stdout)
    for step, data in enumerate(train_bar):
        images, labels = data
        optimizer.zero_grad()
        logits = net(image
        loss = loss_functi
        loss.backward()
        optimizer.step()
```

将模型切换到“训练模式”

- 启用 Dropout（防止过拟合）
- 启用 Batch Normalization (BN) 的统计量更新

```
using 3306 images for training, 364 images for validation.
train epoch[1/3] loss:1.222: 100%|██████████| 207/207 [04:23<00:00,  1.27s/it]
valid epoch[1/3]: 100%|██████████| 23/23 [00:32<00:00,  1.41s/it]
[epoch 1] train_loss: 1.491  val_accuracy: 0.580
train epoch[2/3] loss:1.057: 100%|██████████| 207/207 [04:22<00:00,  1.27s/it]
valid epoch[2/3]: 100%|██████████| 23/23 [00:35<00:00,  1.56s/it]
[epoch 2] train_loss: 1.113  val_accuracy: 0.745
train epoch[3/3] loss:0.902: 100%|██████████| 207/207 [04:13<00:00,  1.23s/it]
valid epoch[3/3]: 100%|██████████| 23/23 [00:35<00:00,  1.55s/it]
[epoch 3] train_loss: 0.908  val_accuracy: 0.775
```

总结

- **第七章 图像分类**
  - 7.1 ImageNet挑战赛
  - 7.2 经典图像分类网络

- AlexNet ➡ 使用ReLu激活函数、引入dropout处理过拟合
- VGGNet ➡ 小卷积核（3*3）的使用
- GoogLeNet ➡ 引入Inception结构
- ResNet ➡ 引入残差学习

  - 7.3 迁移学习：获取预训练知识、知识保留与冻结、 知识迁移与适配、训练策略

- **第八章 目标检测与图像分割**

The End

机器人与信息自动化研究所

Institute of Robotics & Automatic Information System

南開大學

Nankai University